第5章　处理器(CPU)设计

在计算机系统中,中央处理器CPU主要负责获取程序中的指令、译码并执行指令,实现指令们规定的功能。本章主要介绍CPU的结构及设计,重点放在控制器的设计方法,其内容对于深入理解计算机的工作原理有很大的帮助。

5.1　CPU的结构

利用大规模集成电路技术,计算机系统中的控制器、运算器、寄存器组(又称寄存器文件)被集成在一块芯片上,形成了现在称为CPU的中央处理单元。

5.1.1　CPU的功能

CPU是计算机工作的核心部件,用于控制并协调计算机的各个部件执行程序,对数据进行加工及输入/输出。其基本功能主要如下。

1. 指令控制

CPU的首要任务就是对程序执行顺序实施控制,保证程序严格按规定顺序执行。

2. 操作控制

下面对指令执行过程分析将会看到,一条指令的执行需要若干个微操作命令信号,通过这些微操作信号作用于CPU内部及外部的不同部件上,完成指令功能。CPU必须按每条指令功能的要求,产生操作信号并对信号进行控制,完成每条指令所要求的操作。

3. 时间控制

CPU执行每一条指令都有严格的时间顺序。也就是说一条指令的执行过程,什么时刻该产生加在什么部件上的什么操作控制信号是有严格规定的。CPU能够对各操作控制信号的时序进行严格的控制。

4. 数据加工

CPU能够根据指令功能的要求,对数据进行算术、逻辑及其他方面运算和加工处理。对数据的输入、加工处理以及数据的输出是CPU的基本功能。

5. 中断处理

CPU能对其内部或外部的中断(异常)做出响应,进行相应的处理。详见第8章。

6. 其他功能

CPU 能对直接存储器存取(DMA)请求做出响应。能对复位信号(RESET)做出响应并将复位启动的入口地址送入 PC 中。CPU 还能接收外部时钟信号,以便形成其内部的时序控制信号。

5.1.2 构成 CPU 的主要部件

1. 概念框图

计算机的概念框图如图 5-1 所示。可以发现该框图与第 3 章的图 3-30 非常相似,只是为了叙述上的方便,又专门画在图 5-1 上。这是一幅简单的计算机框图,特别强调 CPU 的内部结构,目的是更好地描述传统意义上如何设计 CPU。

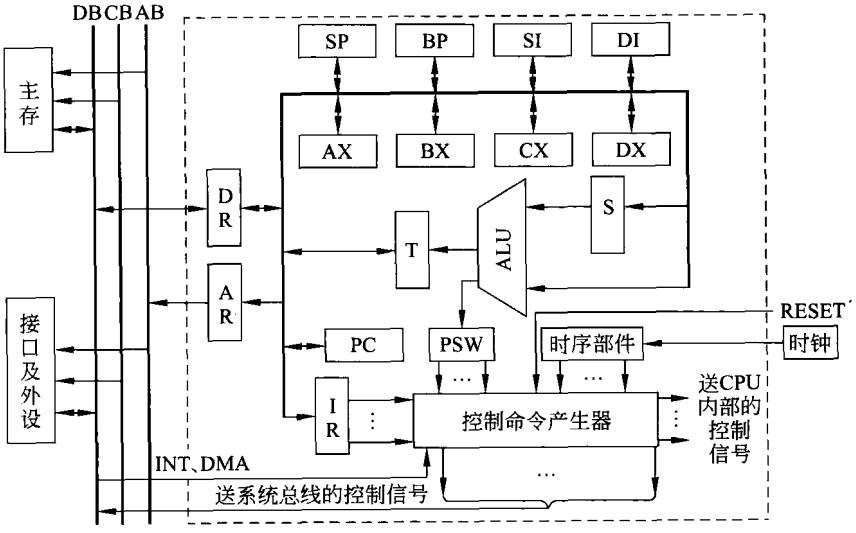


图 5-1 CPU 及计算机简化框图

应当说,这里的 CPU 相对比较简单,本书的目的在于用这样一个简单的 CPU,讲清楚有关的基本概念和基本方法。

2. CPU 的组成部件

如图 5-1 所示的是计算机的简化框图,其中右侧用虚线所包围的部分就是本章要讨论的中央处理器 CPU。这里要讨论的是一个 16 位 CPU,其地址、数据信号均为 16 位,故最大的内存地址空间为 64KB。主要由如下部件构成。

(1) 通用寄存器组

每一个 CPU 的内部都会设置一些通用寄存器,尽管不同的 CPU 所设置的通用寄存器的名称及数量各不相同,但这是 CPU 的重要组成部分。一般称为寄存器组或寄存器文件。

CPU 中包含 8 个 16 位的通用寄存器 AX、BX、CX、DX、BP、SP、SI、DI。它们均为 16 位

寄存器,在功能上略有不同(类似于前一章描述的 8086 的通用寄存器)。

AX、BX、CX、DX 为通用数据寄存器,可用于基本相同的对数据操作功能。

BX、BP 为基址寄存器,主要用于寄存器间接、基址寻址。

SI、DI 主要用于寄存器间接、变址寻址等功能。

SP 为堆栈指针,主要用于堆栈操作。

(2) 运算器

在此 CPU 中,运算器包括如下几种构件。

① 算术逻辑单元 ALU

算术逻辑单元 ALU,能够完成各种算术和逻辑运算。有关 ALU 在前面第 3 章中已做了详细说明,那里的 16 位的 ALU(图 3-27 或 3-28)就可以作为这里的 ALU。

② 单总线

在前面第 3 章中曾说明运算器的三种基本结构,其中单总线结构需要两个暂存器。在现在的 CPU 中,为了简化结构也采用单总线。

③ 两个暂存器

两个 16 位的暂存器 T 和 S,其中暂存器 T 还可以实现移位和循环操作。

把上述部件都归为 CPU 的运算器。尽管有人将通用寄存器组归并在运算器中,在这里还是单独列出。

(3) 控制器

构成 CPU 的另一重要部件就是控制器,由图 5-1 可以看到,控制器主要包括如下几种构件。

① 程序计数器 PC,用于存放下一条要执行指令的内存地址。

② 指令寄存器 IR,存放正在执行的指令。

③ 程序状态字 PSW,存放指令执行结果状态及一些特定的标志。通常,包括进位标志 CF、溢出标志 OF、零标志 ZF、符号标志 SF、中断标志 IF 等。

④ 时序部件,用于产生所需的时序信号。

⑤ 控制命令产生部件,在指令的执行过程中,控制命令产生部件能够根据指令操作码,产生相应的内部及外部控制信号,在这些控制信号的控制下,完成指令的功能。

此外,为了与外部系统总线接口,在 CPU 中设置单向地址寄存器 AR 和双向数据寄存器 DR。还有对外部中断 INT、直接存储器存取 DMA 请求及响应的部件。

在有的 CPU 内部集成有高速缓存(cache)、流水线等部件,它们也应当是构成 CPU 的组成部分。

3. CPU 的数据通路

(1) 数据通路的概念

数据通路是指数据在 CPU 各功能部件之间传送的路径。在这里,数据的概念是广义的,比如,某通用寄存器的数据经内部总线加到 ALU 上,是将总线作为数据通路。同样,PC 送出的地址信号也通过总线传送到地址寄存器 AR 上。而 CPU 从内存读取的指令码也会通过数据通路加到指令寄存器 IR 上。

数据通路描述了信息从什么地方发出,中间经过什么部件,最后传送到哪个部件。而

且,信息每传送一步都需要一定的控制信号(或控制命令)才能做到。执行指令的控制信号是由控制命令产生部件产生,用于建立执行指令的数据通路。

数据通路的功能是实现 CPU 内部运算器、寄存器、控制器等各功能部件间的数据传递。

(2) 数据通路的基本结构

数据通路的基本结构主要有两种形式。

① 总线结构

连接两个以上数字器件的信息通路称为总线,有关总线的详细内容见第 8 章。

由图 5-1 可以看到,在 CPU 内部采用单总线结构,CPU 的各部件均连接在此内部总线上。同时,由于要设计的 CPU 是 16 位的,其内部总线的宽度也是 16 位的。总线上连接的各寄存器也是 16 位的。此外,为简单起见,规定主存按字(16 位)编址,每一主存地址存放 16 位二进制数。

在这种单总线结构中,所有寄存器、ALU、暂存器等部件的输入和(或)输出端均连接在这条单一的 CPU 内部总线上,它们共享一条内部总线。这就产生一个非常重要的问题:任何时刻只允许连接在此总线上的一个部件输出其数据到总线上,其他剩余的部件绝不允许有输出(可以输入)。因为,只要有两个或两个以上的部件将它们的数据同时输出到此总线上,必然使总线产生竞争(也称为争用或冲突)。一旦产生竞争就一定会出错。因此,在单总线结构的 CPU 中,避免总线竞争是特别值得注意的。

为了防止总线竞争,在 CPU 执行指令的过程中,连接在单总线上的部件在向总线输出数据时必须分时操作,即一个部件输出结束置为高阻后,另一部件才能向总线输出数据。当部件没有输出时,其输出端必须置高阻状态。

单总线结构易于发生总线竞争,为防止竞争就需分时操作,分时操作必定降低传输数据的性能。为此,可采用双总线或三总线结构,其结构形式类似于第 3 章图 3-26(b)、3-26(c)。在双总线及三总线结构中,总线竞争的机会要小得多,总线的传输性能可得到提高。显然,这样的总线在构成上也会更加复杂。

② 专用数据通路

在这种数据通路下,CPU 内部各功能部件之间不采用总线相连接,而是设置专用的数据通路,让各部件的数据、地址等信号走自己专用的连接线。这样就可以避免共享总线容易产生的总线竞争,可以更好地提高 CPU 的性能。

构成专用数据通路工作很复杂,工作量很大,要动用更多的硬件资源。

5.2 指令系统设计

从本章的开始,就着眼于一个简单的 CPU 的设计。从提出 CPU 的框图、决定构成 CPU 的主要功能部件到采用单总线构成数据通路。接下来,在本节中,将继续结合图 5-1 所示的 CPU,设计该 CPU 的寻址方式及指令系统。

5.2.1 设计寻址方式

一个 CPU 的寻址方式越多、设置的指令种类越多,则两者组合起来构成的指令就越

多,CPU 的功能也就越强。当然,这必然使 CPU 内部的信号就越复杂,进而 CPU 的内部结构,尤其是控制器也就越加复杂。在这里,设计的思路是在保证 CPU 的基本功能的前提下,尽可能简单一些。

在要设计的 CPU 中,指令有两种格式:单字指令和双字指令两种(在这里规定一个字为二进制 16 位)。即有的指令只用一个字构成,而有的指令则需两个字构成。用于决定寻址方式的是指令中的一个字节,具体规定如图 5-2 所示。

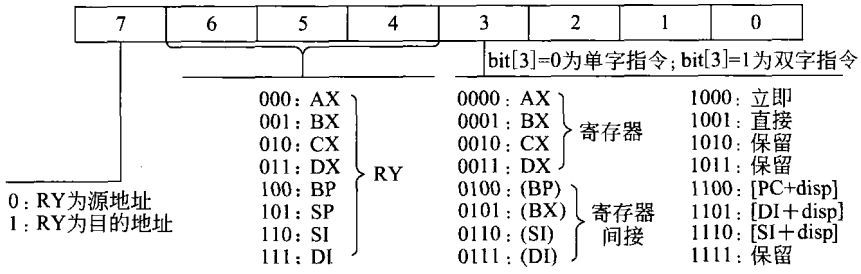


图 5-2 寻址方式的定义

为了简单起见,在图 5-2 中,只规定了 CPU 具有 6 种寻址方式。

(1) 立即寻址。

(2) 寄存器寻址。

(3) 直接寻址。

(4) 寄存器间接寻址。

(5) 相对寻址。

(6) 变址寻址。

利用图 5-2 中字节的 bit[7]来规定 8 个通用寄存器中某一寄存器为源操作数地址,单操作数指令中只须定义一个目的操作数地址;在两个操作数的指令中,bit[7]可指定 RY 是源操作数地址(或目的操作数地址),则另一个由寻址方式所决定的操作数地址必定是目的操作数地址(或源操作数地址)。

即使只有 8 个通用寄存器并且只有 6 种寻址方式,但它们组合起来仍然是比较复杂的,下面将会看到这一点。

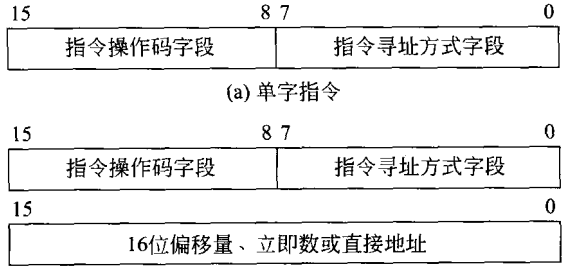
5.2.2 指令的设计

根据不同的寻址方式,指令码的构成有两种形式:单字指令和双字指令。由图 5-2 可以看到,寄存器寻址和寄存器间接寻址的指令只需一个字即可构成;而立即寻址、直接寻址、相对寻址和变址寻址则需要两个字,其中一个字用来表示操作码及寻址方式,另一个字表示立即数、直接地址或偏移量。单字和双字指令的指令码分别如图 5-3(a)和图 5-3(b)所示。

1. 指令的设置

正如第 4 章所提到的,在设计指令系统时,必须考虑到指令系统的有效性、完备性、规整性以及兼容性等诸多方面的要求。在这里所设计的指令系统包括如下几类指令,值得注意

的是为了后面分析指令方便，这里是按操作数的多少来分类的。可以看到，这种分类所设置的指令完全可以涵盖前面第4章中所提到的基本指令。



(a) 单字指令

(b) 双字指令

图 5-3　指令码构成格式

(1) 双操作数指令

双操作数指令设置有如下几种。

与指令 (AND)
或指令 (OR)
异或指令 (XOR)
加法指令 (ADD)
减法指令 (SUB)
带进位加法指令 (ADC)
带进位减法指令 (SBC)
比较指令 (CMP)
传送指令 (MOV)
乘法指令 (MUL)
除法指令 (DIV)

(2) 单操作数指令

单操作数指令设置有如下几种。

加 1 指令 (INC)
减 1 指令 (DEC)
压入堆栈指令 (PUSH)
弹出堆栈指令 (POP)
取反指令 (NOT)
左移指令 (SHL)
逻辑右移指令 (SHR)
算术右移指令 (SAR)
小循环左移指令 (ROL)
小循环右移指令 (ROR)
大循环左移指令 (RCL)
大循环右移指令 (RCR)
调用子程序指令 (CALL)

(3) 转移指令

尽管这类指令也是单操作数指令,在这里单独列出。设置指令有如下几种。

无条件转移指令(JMP)
零转移指令(JZ)
非零转移指令(JNZ)
有进位转移指令(JC)
无进位转移指令(JNC)
大于转移指令(JG)
大于等于转移指令(JGE)
高于转移指令(JA)
高于等于转移指令(JAE)

显然,在上述条件转移中,JC、JNC、JA、JAE 是用于无符号数;而 JG、JGE 则用于带符号数。

(4) 无操作数指令

无操作数指令主要有如下几种。

子程序返回指令(RET)
中断返回指令(IRET)
空操作指令(NOP)
关中断指令(CLI)
开中断指令(STI)
软件中断指令(SWI)
十进制校正指令(DAA)

指令系统中没有设置专门的输入/输出指令,是因为该 CPU 构成的系统采用主存与接口统一编址,用于内存的指令原则上均能用于接口。

2. 指令编码

由图 5-3 可以看出,用于设置指令的操作码字段为指令码的高字节。一个字节的编码共有 256 种,因此,在不采用扩展指令操作码的方法,仍然可以定义 256 种指令。从上面所设置的指令可以看到,一共只有三十几条指令,因此,利用一个字节编码足以定义这些指令。为了留下将来扩展指令的余地,在这里采用扩展操作码的方法来规定指令操作码字段,具体情况如图 5-4 所示。

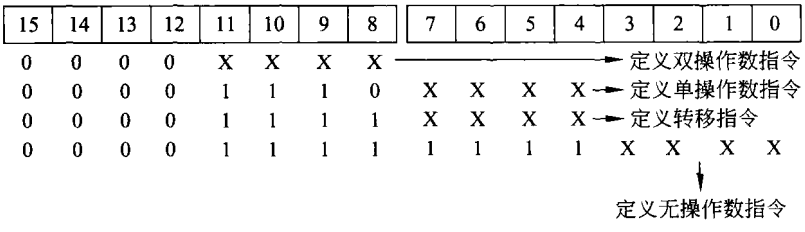


图 5-4 指令操作码的定义

(1) 双操作数指令的编码

双操作数指令为11条,为了简化译码,用指令码高字节的低4位二进制编码表示,也就是用指令字的bit[11]～bit[8]编码来定义这些指令,见图5-4。

由图5-4可以看到,对于上面设置的指来说,指令操作码字段的最高4位bit[15]到bit[12]可固定不变为0000。bit[11]到bit[8]这4位编码有16种,足以定义11条双操作数指令。同时,bit[11]到bit[8]这4位的16种编码中,留下两种编码1110和1111用于扩展操作码。

将来对指令译码时,若bit[15]到bit[12]同时为0000且bit[11]到bit[9]这3位编码不为111时,则必定为双操作数指令,而后根据该指令字的寻址方式,可以判别这是单字指令还是双字指令。由图5-2可以知道,依据不同的寻址方式可以构成单字指令和双字指令。当寄存器或寄存器间接寻址时,也就是寻址字节的bit[3]为0时,必定是单字指令。而当立即、直接、相对和变址寻址方式时,也就是寻址字节的bit[3]为1时,必定为双字指令。

若指定两个操作数的ADD加法指令的操作码字段为00000001,则指令:

```
ADD  AX,[SI]
```

是一条寄存器间接寻址的单字指令,它由16位二进制编码00000001 10000110组成。

若指令:

```
ADD  AX,[2000H]
```

是一条直接寻址的双字指令,该指令由两个16位的字组成,一个为指令码字,一个为直接地址。

```
00000001 10001001   为指令操作码字
00100000 00000000   为直接地址字
```

(2) 单操作数指令的编码

由图5-4可以看到,若bit[11]到bit[8]为1110时,则必定为单操作数指令。由于是单操作数指令,寻址字节的bit[7]到bit[4]这4位对寻址来说已没有用处,恰好用来定义单操作数指令。该4位共有16种编码,足以定义13条单操作数指令。

同样,单操作数指令依据不同的寻址方式可以构成单字指令和双字指令。当寄存器或寄存器间接寻址时,也就是寻址的低字节bit[3]为0时,必定是单字指令。而当立即、直接、相对和变址寻址方式时,也就是寻址的低字节bit[3]为1时,必定为双字指令。假如规定加1指令INC的操作码为00001110 0001,则指令:

```
INC  [BX];为寄存器间接寻址指令,为单字指令,其指令码为:00001110 00010101
```

若指令:

```
INC [DI+ 1000H];是一条变址寻址指令,由两个字构成:
00001110 00011101  为指令操作码字
00010000 00000000  为偏移量字
```

(3) 转移指令的编码

转移指令的编码可用图5-4所示的编码来定义。其操作码在高8位为00001111的情

况下,由寻址字节的 bit[7]到 bit[4]这 4 位来决定。显然,由于转移指令只定义 9 条,4 位编码表示这些指令是有余的。但其中 bit[7]到 bit[4]这 4 位为 1111 的编码留做扩展用,用于扩展无操作数指令。为简单起见,转移指令全部采用相对寻址这一种寻址方式。也就是说这类指令全部都是双字指令,每条指令均由两个字构成。

例如,无条件转移指令 JMP RTPRM,可以规定无条件转移指令 bit[7]到 bit[4]这 4 位编码为 0000。则该指令由 2 个 16 位的字构成。第 1 个为指令操作码字:00001111 00001100。第 2 个字为该指令下一条指令的地址为基准,到达标号 RTPRM 的地址数,转向大地址此数为正,转向小地址时此数为负。这就是相对的偏移量,用一个 16 位带符号的字来表示。

当 CPU 取出这类指令的第 1 个字即指令操作码进行译码时,若发现指令字的 bit[15]到 bit[8]为 00001111 时,必然是一条转移指令。然后检测 bit[7]~bit[4]编码,看是一条什么样的转移指令。

若 bit[7]到 bit[4]这 4 位是 0000,则是一条无条件转移指令,则接着从 PC 的内容所决定的主存读出下一个字,即 16 位的偏移量。这是一个带符号的数,将此时 PC 的内容与该偏移量相加,相加的结果放在 PC 中,则该指令就执行完了。接着执行下一条指令。

若是一条条件转移指令,则接着从 PC 的内容所决定的主存读出下一个字,即 16 位的偏移量。而后需测试条件是否满足。不满足条件接着执行下一条指令。若满足条件,则将此时 PC 的内容与该偏移量相加,相加的结果放在 PC 中,则该指令就执行完了。接着转向执行下一条指令。

(4) 无操作数指令的编码

由图 5-4 可以看到,无操作数指令的编码将操作码扩展到寻址码的最低 4 位,其操作码的编码为 00001111 1111XXXX,这最低 4 位共有 16 种编码,足以用来表示 7 条无操作数指令。

人们常将这种用二进制编码表示的指令称为机器指令。由机器指令构成的语言叫做机器语言。

5.3 CPU 执行指令的过程

5.3.1 指令时序

为了设计 CPU,必须对所设置的每一条指令进行分析,从而弄清楚指令的执行过程。明确了指令的执行过程,才能对 CPU 中最复杂的部件——控制器进行设计。

1. 指令执行过程

(1) 指令周期

CPU 从头到尾执行一条指令所用的时间称为指令周期。从前面指令系统设计可以发现,由于指令的功能不同,执行时间会有所不同。所以不同的指令的指令周期会不一样。

(2) 机器周期

正是由于不同的指令的指令周期不同,研究分析才比较困难。因此,人们把 CPU 通过系统总线对内存的一次读或写操作的时间称为一个机器周期(也有的称为 CPU 周期或总线周期等)。

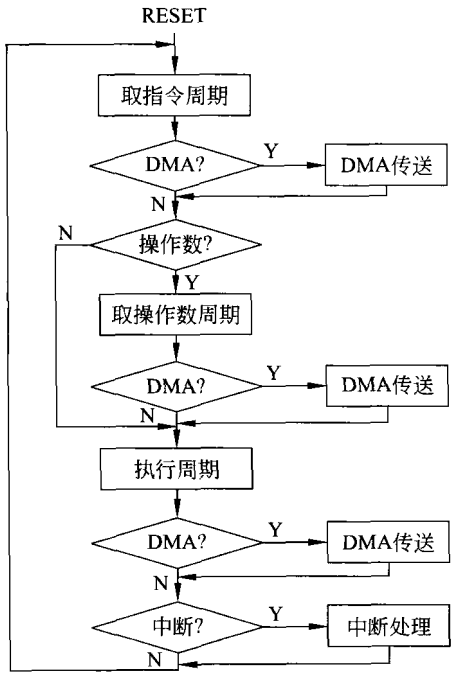


图 5-5 指令执行过程

一条指令的执行可分为三个大的步骤:取指令操作码、取操作数和执行指令,并将这些步骤看做机器周期。在进行 CPU 设计时,通常需要保证每一个机器周期结束需查询有无 DMA 请求,若有请求,则响应 DMA 并等待其传送结束,再进行下一个机器周期。当一条指令执行结束时,要保证先查询有无 DMA 请求,再查询有无中断请求。显然,DMA 请求的优先级比中断请求来得高。上述过程如图 5-5 所示。

由图 5-5 可以看到,CPU 复位之后将会像图 5-5 所示的那样,不知疲倦地一条指令接一条指令地执行下去。每一条指令执行的第 1 个机器周期就是从内存中取指令操作码的周期,这一周期对所有的指令都是一样的。

接下来就是判断这条指令有无操作数,若是一条无操作数指令,则 CPU 将直接进入执行周期。若有操作数,不管是一个还是两个(这里不涉及字串或矢量数据)则进入取操作数的周期。由于操作数多少不同,有一个和两个甚至多个的差别,同时由于寻址方式的不同,取操作数周期的过程是不一样的。

第三个周期是执行周期,这一周也因寻址方式、指令功能的不同而不同。

(3) 时钟周期

如图 5-1 中所示,每个 CPU 必有其时钟。时钟是 CPU 工作的激励源,CPU 是在时钟的激励下执行指令的。时钟可以外接,由 CPU 外部的时钟发生器提供时钟信号,也有的 CPU 将时钟集成在 CPU 内部,则其工作频率将不易改变。

CPU 指令的机器周期是由时钟激励完成的,不同的机器周期所需时钟周期数是不一样的。

2. 时序波形

典型的时序波形如图 5-6 所示。

在这里要强调的是图 5-6 是一种理想化的特殊情况,为的是能用这种简化的时序图说明一些概念。在图 5-6 中一个指令周期是由三个机器周期来实现的,而假定每一机器周期都是由 4 个时钟周期来完成的。

由图 5-6 还可以发现,利用时钟信号和数字电路所学的知识,就能够产生每个机器周期所需要的节拍信号(W 信号)和触发信号(P 信号),也可以产生其他在图 5-6 中未表示的信号。

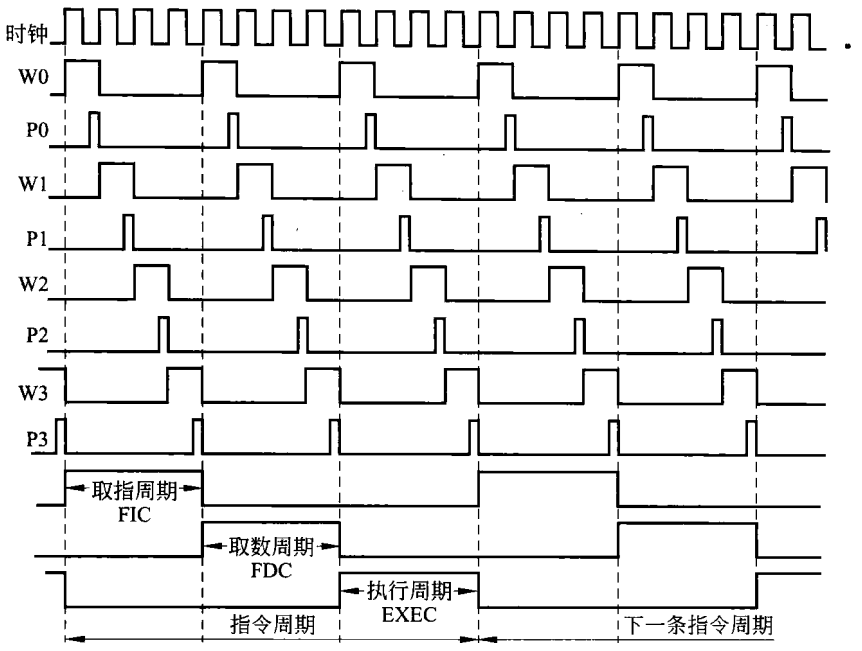


图 5-6 指令执行的时序

CPU 的控制器在不同时刻产生不同的节拍信号、触发信号等一系列信号,在这些信号的控制下,完成每个机器周期。由于这些控制信号出现在机器周期内部,人们还将这些控制信号称做微操作命令。

3. 时序控制方式

从上面的描述可以看到,CPU 执行一条指令实质上是由 CPU 的控制器依据指令的功能,送出一系列的控制信号,最终完成一条指令。指令功能不同,控制器送出的控制信号和送出的时间是不一样的。通常,将如何根据不同指令形成不同系列控制信号所采用的控制方式称为时序控制方式,主要方式如下。

(1) 同步控制

同步控制方式是指系统有一个统一的时钟,所有的控制信号均由这一统一的时钟源产生。具体有 4 种方案。

① 定长的机器周期,定长指令周期

在这种情况下,所有指令的指令周期都一样,都用同样的机器周期来完成,而机器周期的长度(节拍数)也是固定不变的。如图 5-6 那样,一条指令 3 个机器周期,每个机器周期有 4 个时钟周期。因此,每条指令的执行时间都是 12 个时钟周期。

但是,指令的复杂程度差别很大,采用这种方式必须依最复杂的指令设计机器周期,这对简单指令必然造成时间上的浪费。

② 定长的机器周期,变长指令周期

在此方式下,机器周期的长度(节拍数)是固定不变的,而指令可以由一个、二个、三个或更多个机器周期组成。

③ 变长的机器周期，变长指令周期

在这种控制方式下，不同的指令的机器周期长度是不一样的，也就是不同指令用不同节拍来完成。简单的少几个节拍，复杂的情况则增加节拍来延长机器周期。由于机器周期是变长的，则指令周期也一定是变长的。

④ 折中方案

折中方案将上述②和③情况相结合，将大多数指令采用定长机器周期方案，使指令周期在较短的时间内完成。仅仅对那些少量的较复杂的指令，采用变长机器周期。

(2) 异步控制

异步控制方式的系统中没有统一的时钟对信号同步，每条指令执行所需的机器周期数及机器周期的节拍数是不一样的。而且，控制器的工作经常是在信号发出后，等待有关电路做出应答再进行下一步操作。

(3) 联合控制

联合控制是上面同步控制和异步控制相结合的一种控制方式。在此情况下，大部分操作采用某种同步控制方式，而对某些时间上难以确定的操作则采用应答方式。例如，8086 CPU 采用同步②的方式，并在读/写内存或接口时，利用应答信号 READY 来决定机器周期的长短。

5.3.2 指令执行过程分析

为了设计 CPU，必须对所设置的每一条指令进行分析，从而弄清楚指令的执行过程。明确了指令的执行过程，才能对 CPU 中最复杂的部件——控制器进行设计。

1. 指令执行步骤

在图 5-5 中，已经把一条指令的执行分为三个大的步骤：取指令操作码、取操作数、执行指令。由于要进行控制器的设计，在图 5-5 中，需要特别说明的是在取出指令操作码后，在是否取操作数的判断框中隐含着对指令操作码的译码。只有对指令操作码译码后，才能知道这是一条什么指令。只有知道了是一条什么寻址的指令，CPU 才能决定下面的具体操作该怎么做。下面的详细分析中能清楚地看到这一点。

为了分析问题简单起见，详细分析中暂不涉及 DMA 和中断。

2. 详细分析

在对指令进行分析的过程中，必须认真仔细地利用前面的设计内容，尤其是图 5-1 中所规定的 CPU 内部是单总线连接所构成部件，防止在指令执行过程中的总线竞争。同时还必须注意所设置的指令及这些指令所采用的寻址方式。

由前指令设置可以看到，取指令操作码的过程就是从主存中将指令字取出并放在指令寄存器 IR 中。而且这一过程对所有的指令(不管是单字指令还是双字指令)都是一样的。

(1) 取指令操作码阶段

在此 CPU 中，以时钟周期为基本时序单位，对一条指令而言，每一指令执行步骤均由若干个时钟周期来实现。由于各种指令的复杂程度不一样，执行指令所用的时钟周期也不

一样。但是，每条指令的取指过程应是一样的，均由三个时钟周期来完成。取指令操作码的过程如图 5-7 所示。

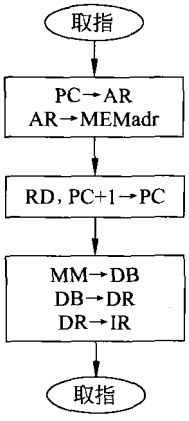


图 5-7 取指过程

由图 5-7 可以看到，取指过程由三步实现：首先将 PC 的内容送入地址寄存器 AR，并且由 AR 输出到系统总线上，作为主存地址 MEMadr；接着送出读控制信号 RD，同时内部程序计数器加 1；第三步就是将从主存读出的指令字（即操作码）送到数据寄存器 DR，DR 再将指令字送到指令寄存器 IR 中。到此取指令操作码的阶段即告结束，假定指令存储器的速度足够快，以上三步用三个时钟周期便可完成。

（2）指令译码取操作数阶段

① 检测指令

由于所设计的 CPU 指令比较多，又有多种寻址方式，不同的指令是靠指令译码来识别进而执行的。因此，指令译码是比较复杂的，可以说一条指令一个样。必须把每条指令的译码过程弄清楚，只有把每条指令的译码过程描述清楚，才能进行后面的设计。

指令的译码过程简化框图如图 5-8 所示，首先是由指令译码器取出指令字，根据前面图 5-4 的定义，若其 bit[15]～bit[9]，若不为 0000111，则指令为双操作数指令；若为 0000111，则再检测指令字的 bit[8]，若 bit[8]=0，则该指令必为单操作数指令；若 bit[8]=1，还需检测 bit[7]～bit[4]，若 bit[7]～bit[4]为 1111，则指令为无操作数指令；若 bit[7]～bit[4]不为 1111，指令定为相对寻址方式的转移指令。

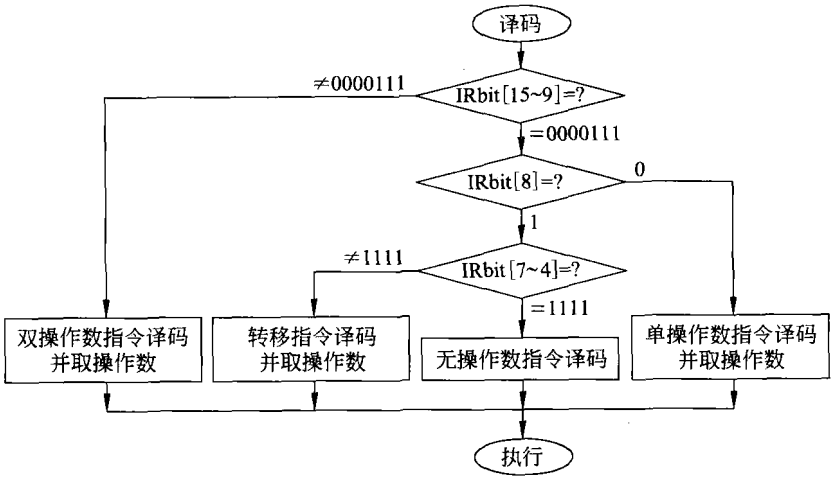


图 5-8 译码取操作数过程框图

根据上面的描述，判断出是哪类指令后，译码过程并未结束，这需要根据不同的指令，进行相应的工作，下将分别予以说明。

② 转移指令译码及取操作数

如前所述，转移指令有 9 种，图 5-8 中转移指令译码部分的详细框图如图 5-9 所示。

在图 5-9 中，为了简化框图，条件转移仅仅画出三条指令，其余的指令由于思路和方法都一样均予以省略。

在转移指令的译码过程中，首先将指令的下一个字，即转移指令的16位偏移量从内存读到CPU的暂存寄存器S中。然后，译码判断若是无条件转移或条件转移满足条件时，译码过程会转向满足条件的执行过程。可以想象，这种情况下指令的执行过程必定是当前的PC与暂存器S中的偏移量相加放入PC中，而后转到该PC所规定的地址上执行新的指令。

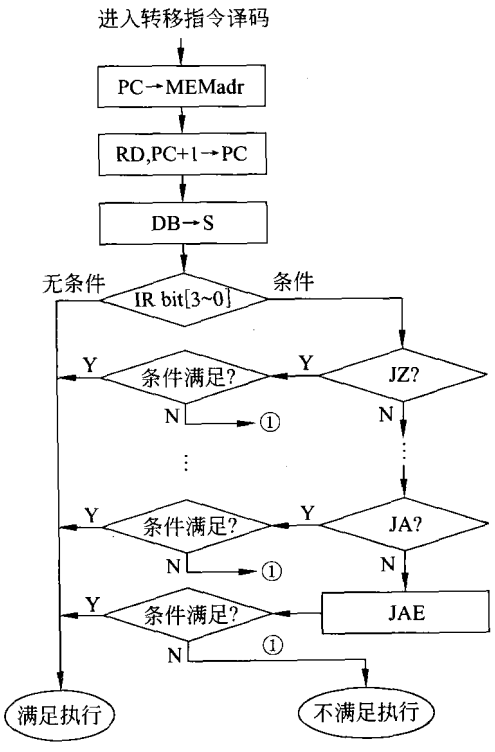


图 5-9　转移指令译码过程

若是条件转移不满足转移条件，则转到不满足条件的情况执行。这种情况下的执行不做任何事而直接利用当前的PC取指，而此时的PC就指向顺序的一条指令。因此，不满足条件的执行就是顺序执行。

③ 非转移指令

非转移指会包括三小类指令，是以指令中所包括的操作数的多少来定义的。在双操作数指令和单操作数指令中，又会根据不同的寻址方式而派生出多条指令。在这里，所要做的工作就是要将每一条指令的译码过程分析清楚。

正如前面一再提到的，即使已经将所设计的CPU简化了许多，但在描述起来仍然感到比较复杂，不可能将每条指令都逐一说明。在下面将以很少的指令说明思路与方法，理解了这样的概念也就不难对其他指令做出解释了。

a）双操作数指令译码与取操作数

对于双操作数的指令，现以如下两条指令为例，说明它们的译码过程。

```
MOV  AX, 4000H
ADD  BX,[DI]
```

双操作数的指令的译码过程如图5-10所示。图中首先根据图5-4的约定，对指令码的bit[11]～bit[8]进行译码，确定是所定义的11条指令中的哪条指令。图5-10中只画出上面两条指令MOV和ADD的译码过程，其余指令则省略。

当指令为MOV指令时，再根据bit[3]＝1确定为立即寻址指令、bit[7]～bit[4]应为1000，即AX为目的地址。于是，以PC的内容为主存地址，由此地址读出立即数4000H，同时PC自动加1。最后，将4000H放入AX中。到此，该指令译码与执行已同时完成。下一个时钟周期将进入下一条指令的取指令码的过程。

对于ADD指令，根据指令字的bit[3]～bit[0]可知，其编码必为0111，即以DI寄存器的内容为主存地址的寄存器间接寻址。同时，由指令字的bit[7]～bit[4]可知，其编码必为1001，即目的寄存器为BX。接下来的过程如图5-10所示，以DI的内容为主存地址，从中读出加法的一个操作数并放在暂存器S中。到此ADD指令的译码过程结束，接着将进入该指令的执行阶段。

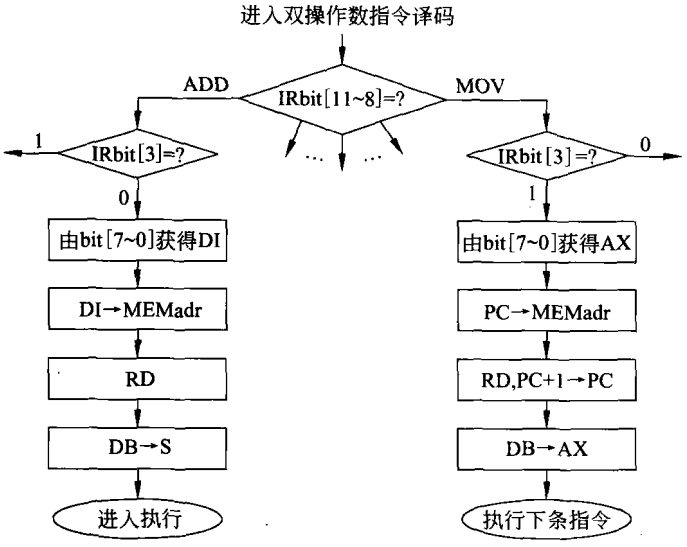


图 5-10 双操作数指令译码过程

b) 单操作数指令译码与取操作数

对于单操作数指令,同样存在着多种指令又有多种寻址方式相组合的问题。为设计控制器,应当把每条指令的每种寻址方式都分析清楚。但在本章中限于篇幅又难以做到。因此,在这里也只取两条指令来说明问题。

```
INC  [BX]
SHR  CX
```

其中,INC [BX]指令是对由 BX 的内容所决定的内存的字内容加 1;而指令 SHR CX 是对寄存器 CX 的内容逻辑右移一次。

单操作数指令的译码过程如图 5-11 所示。

进入单操作数指令译码,首先利用指令字的 bit[7]~bit[4]决定是哪一条单操作数指令。若发现是 INC 指令,则进一步获得指令字的 bit[3]=0,进而得到 bit[2]~bit[0]为 101,即以寄存器 BX 为目标的寄存器间接寻址。于是,将 BX 的内容送入地址寄存器作为主存地址 MEMadr,然后送出读信号,将该存储器的内容读到暂存器中。到此译码结束,后面接着进行的就是该指令的执行过程。

若发现指令为 SHR,利用指令字的 bit[3]~bit[0]可知这是对寄存器 CX 的寄存器寻址指令,接着便将 CX 的内容送到暂存器 S 中。到此译码结束,后面接着进行的就是该指令的执行过程。

c) 无操作数指令译码及执行

对于无操作数指令,由于没有操作数(或许操作数隐含在操作码中),其译码过程同样因指令的不同而不一样。在这里,同样用两条指令予以说明。

```
IRET(中断返回指令)
NOP(空操作指令)
```

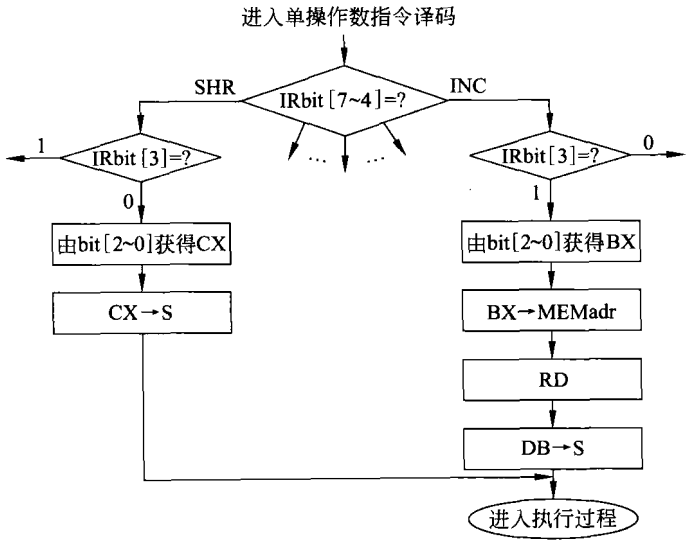


图 5-11　单操作数指令译码过程

无操作数指令的译码过程如图 5-12 所示。

进入无操作数译码，通过对指令字的 bit[3]～bit[0]译码即可获得当前是哪条无操作数指令，接下来便可针对不同的指令实施译码。

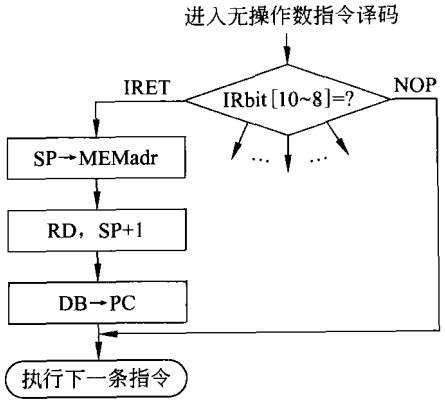


图 5-12　无操作数指令的译码过程

若发现当前的指令是一条空操作指令时，则该指令包括执行就完成了，接下来就进入下一条指令的取指过程，即执行另一条新指令。

对于 IRET 指令，假定在 CPU 响应中断时，CPU 只自动保护 PC 的内容。那么，在执行 IRET 指令时，取出指令字，会发现是无操作数指令，通过对指令字的 bit[3]～bit[0]译码即可获得当前是 IRET 指令，后面的过程译码与执行合并在一起。把保护在堆栈中的断点 PC 的信息重新放回 PC 中。下一条指令必定由此 PC 的内容所决定，即回到被中断的程序，接着中断前的顺序继续执行。

在这里要说明的是，前面描述的译码基本上是针对带有操作数的指令，认为将一个操作数取出来放在某一寄存器中，就是译码阶段，接下来就是指令的执行阶段。显然，这仅仅是一种处理问题的思路和方法。而且，实际上，一条指令的译码和执行是接连发生的、是紧密相连的。因此，要特别强调，前面所提到指令执行的全过程是分开说明的，但指令的执行是顺序进行的，如图 5-5 所描述。

(3) 指令执行阶段

下面将说明指令的执行阶段。不同的指令这一阶段的过程是不一样的。在这里，只说明前面所提到的指令的执行过程。

① 转移指令的执行

首先,接着图 5-9 的无条件转移和满足条件转移的情况继续接着执行阶段,此阶段的过程如图 5-13 所示。

如图 5-13 所示的过程是接在一条转移指令取出指令操作码,而后又取出指令操作数(即偏移地址)后。此时,偏移量已放在寄存器 S 中并加到 ALU 的一端。进入执行阶段,将当前的 PC 的内容加到 ALU 的另一端。然后,将加在 ALU 两端的 PC 与偏移量相加,并将相加的结果放在寄存器 T 中。最后,将 T 中的结果放回 PC。到此,无条件转移指令(或条件转移满足条件)执行结束。下一个 CPU 的状态必定转移到新的 PC 的内容所指向的地址上去取指令、执行指令。

顺便提示,当条件转移不满足条件时,CPU 将依据当时的 PC 的内容顺序执行,如图 5-9 所示的不满足执行。

② 双操作数指令的执行

现在说明双操作数指令的执行过程。其中,立即寻址的传送指令 MOV AX, 4000H,其过程如图 5-10 所示,在译码阶段即已执行结束,在那里就可以直接转到下一条指令的取指阶段。

对于加法指令 ADD BX, [DI],其执行过程如图 5-14 所示。

在如图 5-10 所示的译码阶段,对该指令的译码已取得一个操作数放在寄存器 S 中,并且 S 的输出直接加到 ALU 的一端。

进入执行阶段,再将 BX 的内容通过 CPU 的内总线加到 ALU 的另一端。然后,送出相加命令并将相加结果放在寄存器 T 中。再一个时钟周期,即可将放在寄存器 T 中的相加之和送入 BX。到此,ADD BX, [DI]指令执行结束。下一个 CPU 的状态必定根据 PC 的内容所指向的地址去取下一条指令。

③ 单操作数指令的执行

单操作数指令 INC [BX]及 SHR CX 的执行过程分别如图 5-15(a)和图 5-15(b)所示。

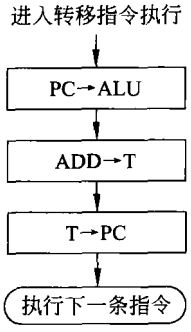


图 5-13 转移指令执行

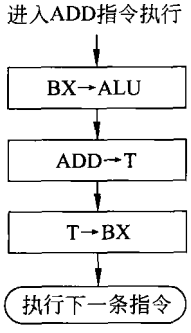
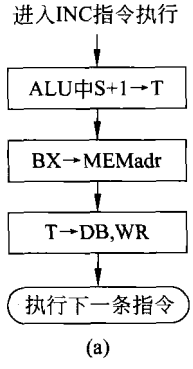


图 5-14 ADD 指令执行



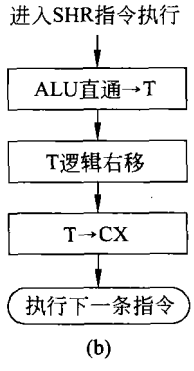


图 5-15 INC 指令及 SHR 指令执行过程

对于 INC [BX]指令,在译码过程中已经将要加 1 的主存单元的内容放在寄存器 S 中,并已加到 ALU 上。在执行过程中(如图 5-15(a)所示),利用命令将 S 寄存器的数据加 1,并将加 1 的结果放在寄存器 T 中。接着将 BX 的内容放在地址寄存器中,形成主存地址

MEMadr。将寄存器 T 的内容送数据寄存器，加在数据总线上，再送出写控制信号 WR，将指令执行的结果写回到原来的存储单元。（图 5-15(a)中 BX→MEMadr 这一步可以省略，因为地址寄存器中仍然放着前面该指令译码时的地址（即 BX 的内容）到此该指令执行结束，接着执行下一条指令。

对于 SHR 指令，在译码过程中已将 CX 的内容放在 S 寄存器中，并且 S 寄存器直接加到 ALU 上。在执行过程中，令 ALU 直通 T 并把 S 的内容直接放寄存器 T 中。下一步是命令寄存器 T 完成逻辑右移功能。最后，将寄存器 T 中的结果放回 CX 中。到此该指令执行结束，接着执行下一条指令。

无操作数指令的译码和执行两步可以合并，前面已经说明，这里不再解释。

5.4 CPU 的部件设计

在本节之前，已经决定了 CPU 的内部结构，设计了 CPU 的指令系统及其寻址方式，并对指令执行过程进行了详细分析。经过前面的一系列工作，在此基础上可以对构成 CPU 的部件进行具体设计。在这节里，将简单说明前面及前期课程已学过的部件，将重点放在控制器的设计上。

5.4.1 熟悉部件的回顾

1. 算术逻辑单元 ALU

算术逻辑单元 ALU 是 CPU 的重要组成部件，有关算术逻辑单元 ALU 的具体内容在第 3 章中做了详细说明。在那里有如何利用 74181 或 74181 和 74182 构成 16 位的算术逻辑单元 ALU 的内容。用那里所描述的构成 ALU 的思路和方法，可以方便地构成如图 5-1 所示的 CPU 中的算术逻辑单元 ALU。

2. 移位寄存器

如图 5-1 所示的 CPU 中，暂存器 T 具有移位功能，能完成前面所设置的移位及循环指令功能。暂存器 T 实际上就是一种并入/并出的移位寄存器。具有这样功能的器件有许多种，前期的数字电路课中会有详细描述。本书前面第 3 章的 3.2 节中有关于该器件的简单说明。利用这样的器件只要在其输出端加上三态门，经三态将其输出接到 CPU 的内总线上，便可以实现暂存器 T。

3. 寄存器及带有三态输出的寄存器

在构成如图 5-1 所示的 CPU 时，会用到许多寄存器及带有三态输出的寄存器。例如，通用寄存器组都是带有三态输出的寄存器。而暂存器 S、指令寄存器 IR 则是一般的寄存器。

(1) 寄存器

在第 3 章的 3.2 节中已介绍了 8 位寄存器。在这里，为了强调其控制信号，16 位的寄

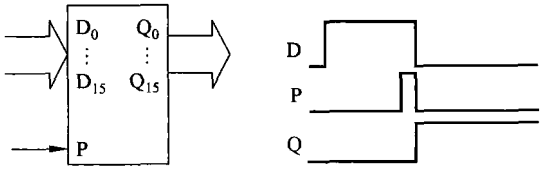
存器框图及信号波形如图 5-16(a)和图 5-16(b)所示。

由图 5-16 可以看到,当将数据加到寄存器的输入端 $D_0 \sim D_{15}$ 上,利用触发脉冲 P,可将输入端 D 的状态寄存在其输出端 $Q(Q_0 \sim Q_{15})$ 上。该输出状态将一直维持到下一个触发脉冲 P 出现、寄存新的输入状态。

该寄存器可用做 S、IR 等寄存器。例如,某一时刻数据寄存器在 W 节拍信号控制下,将指令操作码输出到 CPU 的内部总线上,该信息将加在总线上所有连接部件的输入端。当然也加到 IR 的输入端,此时只在 IR 的 P 端加上触发脉冲,则此指令操作码就一定寄存在 IR 中,并从其输出端输出,加在控制器的指令译码器上。

(2) 带有三态输出的寄存器

同样,带有三态输出的寄存器在第 3 章的 3.2 节中已经介绍过。在这里需要说明的是,利用如图 5-6 所示的某个节拍信号 W 将数据加到寄存器的输入端 $D_0 \sim D_{15}$ 上,利用触发脉冲 P,可将输入端 D 的状态寄存在寄存器的内部,这些状态能否输出由输出允许端 OE 的状态来决定。在这里假定 OE 信号为高电平有效,即当 OE 为高电平时,三态门导通,寄存器内部的状态可输出到芯片外部;当 OE 加上低电平时,其输出为高阻状态。为了说明问题方便,16 位带有三态输出的寄存器如图 5-17 所示。



(a) 框图　　(b) 波形图

图 5-16　16 位寄存器

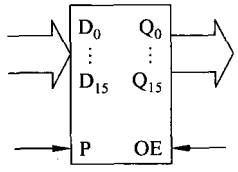


图 5-17　三态输出寄存器

当某个节拍信号 W 将数据加到寄存器的输入端 $D_0 \sim D_{15}$ 上时,利用触发脉冲 P,可将输入端 D 的状态寄存在寄存器的内部,而后,当另一个节拍信号 W 加到 OE 时,其寄存的数据便可输出。

这样的器件可用来构成寄存器组,它们的输入端输出端都接在 CPU 的内部总线上。例如,一个如图 5-17 所示的寄存器接到总线上用做 BX,某时刻用一节拍信号 W 使暂存器 T 的输出三态门导通,则 T 的数据就加到了内总线上。同时,也就加到寄存器 BX 的输入端,此时利用 P 触发脉冲便可将 T 的数据存放在 BX 中。也就是说一个节拍信号 W 和同一节拍的触发脉冲 P,就将 T 的内容存于 BX 中。

当要使 BX 的内容传送到其他寄存器时,必然先用节拍 W 使其内容输出到总线上,要传送到哪个寄存器,就在哪个寄存器上加 P 脉冲。显然,对其他寄存器的工作可以同样的思路去理解。

4. 计数器

对所讨论的 CPU 中的程序计数器 PC、堆栈指示器 SP 以及可以实现加 1 减 1 功能的寄存器,原理上均可用在第 3 章的 3.2 节中也曾提到并入/并出的可逆计数器、再在其输出端接上三态门来实现。例如,用 16 位并入/并出的计数器,并在计数器的输出接三态门,将其输入和三态门输出接到 CPU 的内部总线上,便可以构成程序计数器 PC。只是该 PC 在

CPU 复位时能使其输出置为 CPU 的复位启动地址。

当某寄存器(假定为 T 暂存器)的内容送 PC,则可用 W 节拍信号打开 T 的三态门,T 的数据就加到了 CPU 的内总线上,这时用 P 脉冲加到 PC 的预置端就能将数据写入 PC。同样,当要使 PC 的内容传送到其他寄存器(例如 AR)时,必定先用节拍 W 打开其输出三态门,使其内容输出到总线上,现要传送到地址寄存器 AR,就在地址寄存器 AR 上加 P 脉冲。然后,利用另一 P 脉冲就可以使 PC 加 1。显然,对其他寄存器的工作可以同样的思路去理解。

5.4.2 指令执行的详细说明

1. 组合逻辑控制器的构成

(1) 框图

图 5-1 中已说明控制器构成的 5 个组成部分,在这里针对组合逻辑控制器说明它的组成,其框图如图 5-18 所示。

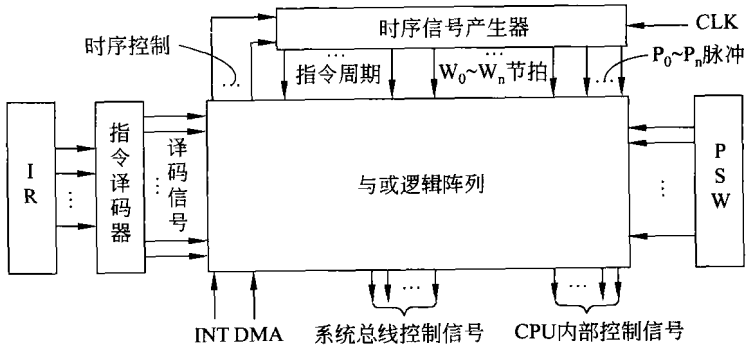


图 5-18 硬件组合逻辑控制器框图

(2) 组成

由框图 5-18 可以看到,组合逻辑控制器是由如下部件构成的。

① 时序信号产生器

该产生器产生如图 5-6 所示的三类信号。

a) 指令周期,包括取指令码周期 FIC、取操作数周期 FDC 和执行周期 EXEC。

b) 节拍信号 W_0～W_3。

c) 触发脉冲 P_0～P_3。

为了使问题简单一些,时序控制采取同步方式,一机器周期由 4 个时钟周期来完成。

② 指令译码器

如上节所述,利用指令译码器,可根据不同的指令操作码和不同的寻址方式产生不同的信号,再经与或逻辑阵列产生相应的控制信号。

③ 程序状态字 PSW

程序状态字 PSW 在 5.1 节中已说明,它里面存放着 CPU 执行一条指令后所处的状态。在条件转移指令执行过程中,会根据指令执行结果所产生的状态来决定程序的转向。

④ 与或逻辑阵列

利用与或逻辑阵列产生每一条指令执行所需要的全部控制信号。很显然,这部分是由与门、或门等硬件电路构成,是很复杂的。尤其当指令多、寻址方式多的情况下就会更加复杂。

2. 微操作命令及其安排

(1) 微操作命令

在前面的描述中已经看到,为了方便分析和设计控制器,将指令分成若干机器周期,再将机器周期分成几步,每一步只完成一种最基本的操作。人们把这种最基本的操作称为微操作。实现微操作的控制信号称为微操作命令。例如,此前提到的 PC→IB,BX→IB 等操作都是微操作,而实现这些微操作的控制信号 PC_{out},BX_{out} 等信号都是微操作命令,它们就是上面提到的打开三态门的信号。同样,触发信号 P 脉冲是微操作命令,在下面常用 PC_{in},BX_{in} 表示。显然,基本功能为微操作,实现微操作的控制信号是微操作命令。

下面将要说明指令执行过程中微操作命令的安排。

(2) 微操作命令的安排

一个指令周期,最终是由一系列微操作命令来完成的。前面已经分析过了,在这里接着本章前面的内容更加具体地进行叙述。为了简单起见,规定一个机器周期由 4 个节拍来实现,如图 5-6 所示。

① 取指令码周期 FIC

每条指令的执行,首先就是从主存中取指令码,这个过程对所有的指令都是一样的。可以说这一取指令码过程是共用的。其过程如下。

W_0　　PC_{out}, AR_{in}

W_1　　RD, PC+1

W_2　　MD→DB, DR_{in}

W_3　　DR_{out}, IR_{in}

在上述过程中,PC_{out} 是将 PC 的内容输出到 CPU 的内部总线 IB 上,这是一种微操作命令,就可直接用 W_0 加到 PC 的输出三态门上实现该操作。AR_{in} 也是一种微操作,其功能就是将出现在 IB 的 PC 的内容锁存在地址寄存器 AR 上。该微操作可用触发脉冲 P_0 来实现。

从上面所描述的过程可以看到,取指周期由 4 个节拍来实现。W_0 节拍前面已做了解释,其他 3 个节拍读者可自行理解。

当最后一个节拍结束时,应当判别有无 DMA 请求,此处不做说明。没有 DMA 请求或 DMA 传送结束,则根据指令有无操作码,决定启动取操作数周期 FDC 或执行周期 EXEC。

② 取操作数周期 FDC

由于指令有不同个数的操作数,也可以没有操作数,再加上不同的寻址方式。在上一节分析时已经强调,这一周期不同的指令是很不一样的。在这里要说明的是,为了设计控制器,必须把所有寻址方式的所有指令的微操作命令安排妥当。

显然,对于前面所要设计的 CPU,由于指令比较多,寻址方式也比较多。要将每一条指令取操作数周期的微操作命令都写出来,作为教科书来说就太繁杂了。不过请读者记住,要真正用现在这种思路设计组合逻辑控制器,就必须将所有指令、所有寻址方式的所有微操作

命令安排全都写出来。

在这里，为了说明问题，只将前面提到的几条指令取操作数的微操作命令写在下面。

a) MOV AX，4000H

该指令的取操作数周期过程如下。

W_0 PC_{out}，AR_{in}

W_1 AR→AB，RD，PC+1

W_2 MD→DB，DR_{in}

W_3 DR_{out}，AX_{in}

这里要说明的是，该指令取操作数周期结束，该指令执行也就结束。利用最后的节拍启动下一条指令的 FIC 周期。

b) ADD BX，[DI]

该指令的取操作数周期过程如下。

W_0 DI_{out}，AR_{in}

W_1 AR→AB，RD

W_2 MD→DB，DR_{in}

W_3 DR_{out}，S_{in}

在此取操作数的周期结束时，就将以 DI 的内容为内存地址的存储器单元的内容读出并放在暂存器 S 中，其输出加到了 ALU 的一边，接着就进行下一步执行周期 EXEC。

c) 转移指令

在这里写出无条件转移和满足条件的条件转移的取操作数微操作命令安排。不满足条件顺序的执行前面已经提到，这里不再说明。

W_0 PC_{out}，AR_{in}

W_1 AR→AB，RD，PC+1

W_2 MD→DB，DR_{in}

W_3 DR_{out}，S_{in}

将指令操作数即偏移地址取出放入暂存器 S。

d) INC [BX]

该指令的取操作数周期过程如下。

W_0 BX_{out}，AR_{in}

W_1 AR→AB，RD

W_2 MD→DB，DR_{in}

W_3 DR_{out}，S_{in}

在此取操作数的周期结束时，就将以 BX 的内容为内存地址的存储器单元的内容读出并放在暂存器 S 中，其输出加到了 ALU 的一边，接着就启动下一步执行周期 EXEC。

e) SHR CX

该指令的取操作数周期过程如下。

W_0

W_1

W_2

W_3 CX_{out}, S_{in}

此取操作数周期只需一个节拍,那就将其他3个节拍空着。

在此取操作数的周期里,只将CX的内容送出,经内总线IB放在暂存器S中,其输出加到了ALU的一边,接着就启动下一步执行周期EXEC。

在前面提到的无操作数指令,对空操作指令NOP来说,取出指令操作码则该指令执行结束,可启动新的取指周期。对于中断返回指令IRET,取指周期结束可启动下一步执行周期EXEC。

同样,当本周期的最后一个节拍结束时,应当判别有无DMA请求,此处不做说明。没有DMA请求或DMA传送结束,则启动执行周期EXEC。

③ 执行周期EXEC

对应上面的几条指令,它们的执行周期微操作命令信号安排如下。

a) MOV AX, 4000H

该指令取指令操作数与执行周期合在一起,前面已经说明。

b) ADD BX, [DI]

该指令的执行周期过程如下。

W_0 S→ALU

W_1 BX_{out}→ALU

W_2 ALU,ADD→T, T_{in}

W_3 T_{out}, BX_{in}

在此周期里,ALU将放在S中的操作数与BX的内容相加送T,T输出放回BX中。

c) 转移指令

在这里写出无条件转移和满足条件的条件转移的执行周期微操作命令安排。

W_0 S→ALU,

W_1 PC_{out}→ALU

W_2 ALU,ADD→T, T_{in}

W_3 T_{out}, PC_{in}

将PC的当前值与偏移地址相加,构成目的地址放入PC。

d) INC [BX]

该指令的执行周期过程如下。

W_0 S→ALU

W_1 ALU+1→T, T_{in}

W_2 T_{out}, DR_{in}

W_3 DR→DB,WR

在此周期中,将上面周期取得的操作数经ALU加1,再送到数据寄存器DR中。此时,AR中仍存有操作数的地址,送时就将DR出的数据写回原来的地址。

e) SHR CX

该指令的执行周期过程如下。

W_0 S→ALU

W_1 ALU 直通→T, T_{in}

W_2 T 逻辑右移一次

W_3 T_{out}, CX_{in}

f) IRET

中断返回指令由取指周期直接进入执行周期，其过程如下。

W_0 SP_{out}, AR_{in}

W_1 AR→AB, RD, SP+1

W_2 MD→DB, DR_{in}

W_3 DR_{out}, PC_{in}

g) 中断响应过程 INTA

有关中断的详细内容请见第 8 章。在这里仅介绍最简单的情况：对中断请求响应时，CPU 仅保护 PC 的内容，并且由硬件提供该中断的中断向量。其过程如下。

W_0 SP_{out}, AR_{in}

W_1 AR→AB, SP-1

W_2 PC_{out}, DR_{in}

W_3 DR→DB, WR

W_0 0→IF, IF_{in}

W_1

W_2

W_3 IV_{out}, PC_{in}

中断响应过程需两个机器周期，第 1 个机器周期保护断点 PC 的内容入堆栈。第 2 个机器周期关中断(使 IF=0)，并将特定的该中断的中断向量 IV 放入 PC，并启动取指周期。则下一个机器周期 CPU 必定进入该中断的中断处理程序入口开始执行。

5.4.3 组合逻辑控制器设计

1. 微操作命令的逻辑表达式

在这里必须再次强调，为了获得微操作命令的逻辑表达式，必须将所有寻址方式的所有指令的微操作命令安排写出来。正如前面已经提到的，只有将每一条指令每一个机器周期的微操作命令排列出来，才能进行下一步的设计工作。但是，设计 CPU，全部写出微操作命令安排将是非常繁杂的。因此，必须清楚地知道，真正要设计控制器，就必须这样做。

现在，假定已经将要设计的 CPU 的全部微操作命令安排都已写出来了，下一步工作就是写出每一个微操作命令的逻辑表达式。

$PC_{out}=FIC \cdot W_0+FDC \cdot IR(bit_{15\sim9}\neq0000111 \cdot bit_3=1) \cdot W_0+FDC \cdot IR(bit_{15\sim8}=00001111 \cdot bit_{7\sim4}\neq1111) \cdot W_0+INTA \cdot W_2+EXEC \cdot IR(bit_{15\sim9}=0000111 \cdot bit_8=1 \cdot bit_{7\sim4}=1111) \cdot W_1+\cdots$

$AR_{in}=FIC \cdot P_0+FDC \cdot IR(bit_{15\sim9}\neq0000111 \cdot bit_3=1) \cdot P_0+FDC \cdot IR(bit_{15\sim9}\neq$

$0000111 \cdot bit_7=1 \cdot bit_2=1) \cdot P_0+FDC \cdot IR(bit_{15\sim8}=00001111 \cdot bit_{7\sim4}\neq1111) \cdot P_0+FDC \cdot IR(bit_{15\sim8}=00001110 \cdot bit_2=1) \cdot P_0+EXEC \cdot IR(bit_{15\sim4}=000011111111 \cdot bit_{7\sim4}=$IRET 指令$) \cdot P_0+INTA \cdot P_0+\cdots$

$RD=FIC \cdot W_1+FDC \cdot IR(bit_{15\sim9}\neq0000111 \cdot bit_3=1) \cdot W_1+FDC \cdot IR(bit_{15\sim9}\neq0000111 \cdot bit_7=1 \cdot bit_2=1) \cdot W_1+FDC \cdot IR(bit_{15\sim8}=00001111 \cdot bit_{7\sim4}\neq1111) \cdot W_1+FDC \cdot IR(bit_{15\sim8}=00001110 \cdot bit_2=1) \cdot W_1+EXEC \cdot IR(bit_{15\sim4}=000011111111 \cdot bit_{7\sim4}=$IRET 指令$) \cdot W_1+\cdots$

⋮

以上只写出三种微操作命令的逻辑表达式,而且没有写全。只有在所有指令的微操作命令过程都写出后,再根据指令操作码和寻址方式,可把每一条微操作命令的表示式写全。下一节将会看到,在所讨论的 CPU 中,需要用到 50 余种微操作命令。

即使这样,从上面简要写出的不完全的表达式,也可以看到如下几点。

(1) 所有的微操作命令都可以用与或门来实现。由于指令和寻址方式较多,与或表达式也比较复杂。

(2) 在不同的机器周期里、不同的节拍,会用到同样的微操作命令。因此,必须将所有的微操作命令的安排都写出来,才不至于遗漏。

(3) 不同的指令、不同的寻址方式、条件转移指令的条件都是微操作命令的形成条件。上面的表达式中可以看这种情况(注:未包括条件转移指令的条件)。

(4) 前面如图 5-6 所形成的 FIC、FDC、EXEC、$W_{0\sim3}$、$P_{0\sim3}$信号以及指令译码器和 PSW 的信号共同加到与或门上,形成所需要与或逻辑阵列,产生 CPU 工作所需要的所有信号,这也是框图 5-18 所表示的意图。

2. 微操作命令的逻辑化简

上面提到,要设计组合逻辑控制器,就必须将所有用到 50 余种微操作命令的逻辑表达式写出来。写出后并不依照这些表达式直接去构成与或门。因为这太复杂,而且有许多信号是重复的。

鉴于这种情况,就需要将这些微操作命令的逻辑表达式进行综合化简。

3. 构成与或逻辑阵列

在将所有微操作命令逻辑综合化简的基础上,便可以构成与或逻辑阵列,以便产生 CPU 所需要的所有内、外部控制命令。在计算机发展的早期,人们用与门、或门等门电路来实现。可想而知,构成此与或逻辑阵列是相当麻烦的。

当出现简单的可编程逻辑器件 PLD(PAL、PLA、GAL 等)后,人们用它们实现与或逻辑阵列,相对要简单一些,但还是很麻烦的。

今天,人们早已不用这样的器件来设计控制器,要用的方法和手段将在本书的最后一章中详加说明。这里,希望读者理解 CPU 设计的思路和分析方法,这是重要的、是最基本的概念。

5.4.4 微程序控制器设计

1. 微程序控制器原理

(1) 由来

在上面的讨论中，规定 CPU 执行指令的时序分为三个机器周期，每个机器周期分为 4 个节拍，每个节拍为一个时钟周期。这是一种分析和解决问题的方法。

完全可以将一条指令的执行过程顺序地由多个步骤(或称为状态)来完成。也就是将每一条指令的执行过程分解为若干个状态，每一状态的操作完成时根据情况启动下一个状态。当一条指令最后一个状态完成时，再启动执行下一条指令的起始状态。

首先，写出 5.4.3 小节中两条指令的过程。

① ADD BX, [DI]

a) PC_{out}, AR_{in}

b) AR→AB, RD, PC+1

c) MD→DB, DR_{in}

d) DR_{out}, IR_{in}

e) DI_{out}, AR_{in}

f) AR→AB, RD

g) MD→DB, DR_{in}

h) DR_{out}, S_{in}

i) S→ALU

j) BX_{out}→ALU

k) ADD→T, T_{in}

l) T_{out}, BX_{in}

② INC [BX]

a) PC_{out}, AR_{in}

b) AR→AB, RD, PC+1

c) MD→DB, DR_{in}

d) DR_{out}, IR_{in}

e) BX_{out}, AR_{in}

f) AR→AB, RD

g) MD→DB, DR_{in}

h) DR_{out}, S_{in}

i) S→ALU

j) ALU+1→T, T_{in}

k) T_{out}, DR_{in}

l) DR→DB, WR

由上面所描述的指令执行过程可以看到，一条指令的执行过程可以分解为若干更微小

的基本操作,并称这种基本操作为微操作。控制完成微操作的命令称为微命令。能够产生一个或多个微命令的二进制编码就称为微指令。通过将完成一条机器指令的一系列微指令存入存储器中,然后再按顺序依次读出执行,从而完成指令的功能。并称这些完成机器指令功能的一系列微指令构成的程序称为微程序。执行一条指令所对应的一段微程序,就能完成这条指令的功能。利用微程序控制指令的执行这就是微程序控制器的基本思路。

(2) 微程序控制器的构成

微程序控制器的构成框图如图 5-19 所示。

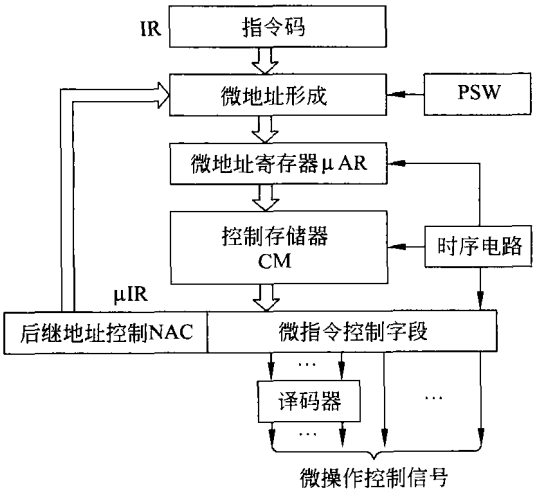


图 5-19 微程序控制器的构成框图

① 控制存储器 CM

控制存储器 CM 用于存放 CPU 所能执行的所有指令的微程序。一旦指令确定,它所对应的一段微程序也就确定了,不会随便改动。因此,控制存储器 CM 都是只读存储器 ROM。并且,希望该存储器的读出速度尽可能快。只有控制存储器 CM 读出速度快,执行一条指令所对应的一段微程序也就会更快,则 CPU 的速度就更快一些。

可以想象,在微程序控制器里增加一条指令就是在控制存储器 CM 中增加存储这条指令的一段微程序,做起来比较容易。若要在组合逻辑控制器里增加一条指令,那就复杂多了。

② 微地址形成电路

如图 5-19 所示,该电路根据指令寄存器中指令操作码 OP 及指令的寻址方式,根据 PSW 的标志以及后继地址控制字段 NAC 的编码等因素,形成控制存储器 CM 的地址。在图 5-19 中未能画出,当有外部复位 RESET 时,该电路必定能产生固定的微地址,从那里开始执行取指令操作码的微程序。

③ 微地址寄存器 μAR

该寄存器用来存放微地址形成电路产生的微地址,也就是控制存储器 CM 的地址。

④ 微指令寄存器 μIR

微指令寄存器 μIR 用来存放由控制存储器 CM 读出的微指令。由图 5-19 可以看到,微指令包括两个字段。

a）微指令控制字段

该字段经译码或不经译码产生 CPU 执行指令所需要的微操作控制信号。有关微指令控制编码的问题，下面将专门讨论。

b）后继地址控制字段 NAC

该字段的不同编码用以表示后继地址的产生方式，主要有如下几种。

顺序执行，即后继地址为当前微地址加 1。

无条件转移，转移地址可由该字段直接给出。

条件转移，二分支的转移，即满足条件转移，不满足条件顺序执行。

多分支转移，根据指令操作码 OP 或寻址方式 MOD 进行的多分支转移。

有关微地址的形成，下面将做详细讨论。

⑤ 时序电路

微程序控制器中的时序电路相对比较简单，只要产生节拍信号和触发脉冲，通过微指令译码就可形成微操作命令。

2. 微地址形成

微程序控制器中，一个重要部件就是微地址形成电路。其结构还与微指令的结构有关，具体有如下几种情况。

（1）两地址格式（断定方式）

最简单的方法是在每条微指令 NAC 域中提供两个地址字段和地址选择控制位（字段）AC，如图 5-20 所示，地址 1 和地址 2 分别提供下条命令的顺序地址和跳转地址。可以灵活地生成跳转地址以及确定下条微指令的地址是选择地址 1（顺序地址）还是地址 2（跳转地址）。

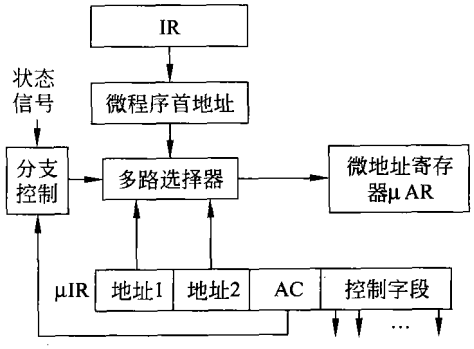


图 5-20　两地址格式原理框图

如图 5-20 所示，利用地址选择控制位 AC 及 CPU 的状态标志使分支逻辑产生地址选择信号，依据此信号，多路选择器可以选择顺序地址、跳转地址或微程序首地址之一作为下一条微指令的地址送入微地址寄存器中。微程序首地址逻辑电路可将输入的指令操作码映射为微指令地址（微程序首地址）并输出。

若控制存储器有 512 个存储单元，则控制存储器地址（微地址）为 9 位，那么两地址格式提供的微指令地址域长度应为（2×9＋m）位，m 为地址选择控制位 AC 的位数。

（2）单地址格式（计数方式，增量方式）

正如在内存中存放程序一样，在控制存储器中也是将顺序执行的微指令存放在连续的控存单元中。因此，顺序微地址通过当前微地址加 1 即可获得，计数器正适合完成此功能。参照 CPU 利用程序计数器（PC）提供顺序的内存地址，在控制器中引入了微程序计数器（μPC），利用微程序计数器具备的加 1 计数的能力，产生下条顺序微地址。

这样，在微指令中只需要一个地址字段提供跳转的微地址即可，微指令格式如图 5-21 中微指令寄存器 μIR 所示，地址字段仅提供下条跳转地址，而下条顺序地址则通过 μPC 加

1 获得。同样,在地址域中仍设置了地址选择控制位(字段)AC,用它来确定何时需要使用跳转地址及如何生成实际的跳转地址。

如图 5-21 所示微程序控制器根据微指令 AC 字段提供的分支跳转信息和系统当前的状态信息生成分支跳转地址加载允许信号。在微程序执行过程中,若该信号有效,多路选择器选择将微指令地址字段的跳转地址输出并加载至 μPC 中;若跳转地址无效或未执行新的微程序,则多路选择器不输出,由 μPC 加 1 的结果作为下条微指令的地址提供给控制存储器。

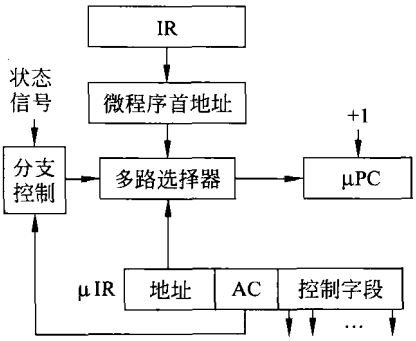


图 5-21 一地址格式原理框图

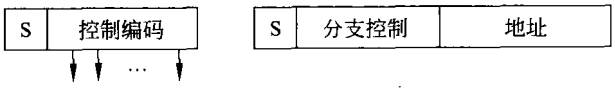
在图 5-21 的分支控制逻辑中,μPC 具有独特的功能。首先,它是一个典型的计数器,在每一个微指令周期(节拍),时序电路向 μPC 发出"加 1"控制信号,μPC 实现自动加 1 操作,如果此时多路选择器无输出,μPC 的内容就是下条顺序的微地址。其次,它也像所有的计数器一样可以进行计数初值的加载,当需要执行与机器指令相应的一段新微程序时,微程序的首地址被加载至 μPC,使 μPC 指向微程序在控存中的首地址。之后可以依据此首地址并通过不断地加 1,获得微程序段内其他微指令在控存中存放的顺序地址。而当微程序需要分支跳转时,分支跳转地址被加载至 μPC。这样就可以依据此地址重新从控存中获得一组微指令。对 μPC 的这种加载功能使 μPC 完全可以取代 μAR,所以图 5-21 中没有出现 μAR。

引入硬件逻辑并不复杂的 μPC 代价极低,不但 μPC 利用率高,而且微指令长度被有效地缩短。

(3) 可变微指令格式

在微程序中,多数情况下是顺序程序。当执行顺序微程序段时,微指令的地址域信息实际是无用的。也就是说,在微程序执行的绝大多数时间里,微指令中地址信息是多余的。如果可以让微指令在顺序执行时只提供控制信号的产生,需要分支时再提供跳转地址,这样就可以保证在任何微指令执行时不存在无用信息。这就是提出可变微指令格式的依据。

为此,设计了两种微指令格式,分别如图 5-22(a)和图 5-22(b)所示。



(a) 控制微指令 (b) 转移微指令

图 5-22 两种不同的微指令

由图 5-22 可以看到,两种微指令的功能是完全不一样的。其中图 5-22(a)只用于形成各种微命令;而图 5-22(b)则只用于微程序的分支转移。两种指令用标志位 S 来决定。例如,S=0 时为控制微指令,而当 S=1 时,则为转移微指令。

可变微指令的工作原理可由图 5-23 来说明,当从控制存储器中读出控制微指令时,利用其标志 S=0 控制门电路输出,译码产生相应的控制信号。同时,使 μPC 自动加 1。

当从控制存储器中读出转移微指令时,由于其标志 S=1,门电路没有输出。在它控制

下，利用状态及分支控制产生后继地址。

（4）多分支地址形成

不同的指令编码表示不同的指令，同样的指令又有多种寻址方式。在微程序控制器的控制存储器 CM 中，不同的指令、不同的寻址方式必定对应一段不同的微程序。这些微程序在 CM 的地址也必定不相同。当 CPU 从内存取出一条指令后，对应这条指令的执行，就会面临多分支转移的问题。

诚然，多分支可以通过二分支来实现，也可以采用如图 5-24 所示的原理框图来实现。

由图 5-24 可以看到，不同的指令，其操作码不一样，可以将操作码作为 ROM1 的地址，在该地址中存放着该指令微地址的一部分。指令的寻址方式及状态信号也用于作为 ROM2 的地址，在该地址中存放着该指令微地址的另一部分。将这两部分地址结合在一起，便可以构成这条指令微程序的首地址。

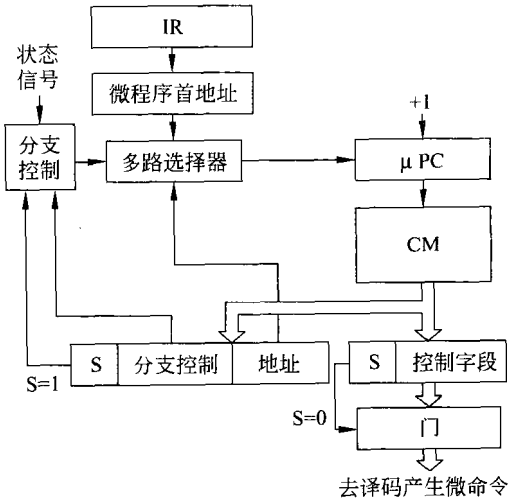


图 5-23　可变微指令格式原理框图

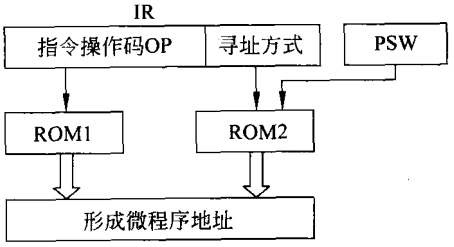


图 5-24　多分支地址形成

3. 微指令控制域编码

正如前面提到的微指令控制域编码要详细加以说明，它是构成微指令重要的组成部分。微指令在执行时，就是由该控制域的编码产生微命令。通常，将微指令分为水平型微指令和垂直型微指令。水平型微指令可以使多个控制信号同时有效，达到使多个微操作同时发生的效果。垂直型微指令类似于机器指令，它利用微操作码的不同编码来表示不同的微操作功能。

（1）水平型微指令

① 直接表示法

前面已经提到，对于如图 5-1 所示的 CPU，执行其全部指令需要 50 余种微操作控制信号（也就是微命令）。在微指令的控制域字段中，直接表示法就是用二进制位表示一种微命令。

例如，若计算机系统需要 n 个控制信号，则控制域字段长度应为 n 个二进制位。就前述 CPU，用直接表示法表示的微指令控制字段如图 5-25 所示。

在图 5-25 中,控制字段的每一位定义为一种微命令,如图中的 PC_{out}、BX_{out}信号等。微命令有多少个,控制字段就有多少位。当某位设定为"1"时,表示该位定义的控制信号有效,为"0"时,表示该位定义的控制信号无效。这样,就可以利用一条微指令,同时使多个控制信号有效。

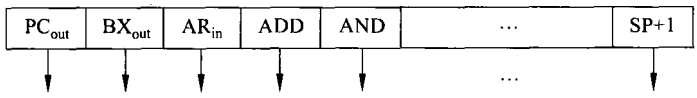


图 5-25 直接表示法的控制字段

采用这种编码方式时,由于控制信号直接从微指令控制域的某位获得,所以在图 5-19 示意的微程序控制器中,不需要译码器。将微指令寄存器的控制域各位直接引出控制器,就获得了计算机系统的全部控制信号,所以这种设计法也称做不译码法或直接控制法。显然,这种设计可以快速地产生控制信号。

可以想象,这种编码构成的微指令控制字段很长而且利用率是非常低的。机器指令执行过程的每一步,只需少量的几个微命令。因此,每一条微指令的控制字段中只有少数几个 1,绝大多数位都是 0(该位无效)。而微指令越长,则 CM 的容量就越大,代价也越高。因此,在水平型微指令的控制字段编码中常用下面的方法。

② 字段译码法(字段编码)

在计算机系统运行时,可以在同一个时间有效的控制信号称为相容信号,具有相容性。也就是说这些信号可以同时出现;不能在同一个时间有效的控制信号称为互斥信号,具有互斥性。也就是说这些信号不允许同时出现。相容的控制信号可以在同一条微指令中设置为"1",互斥的控制信号必须在不同的微指令中设置为"1"。

如图 5-1 所示的单总线结构的 CPU 来说,下列情况下,控制信号必定是互斥的。

a) 所有向内总线 IB 的输出信号必是互斥的,例如,PC_{out}、BX_{out}等。这些信号一旦同时出现,必定引起总线竞争,一定会造成严重后果。

b) 多功能器件的诸多功能必是互斥的。例如,ALU 某一时刻只能完成一种功能,做加法时不可能同时做减法。移位器件不可能同时进行左移和右移。

c) CPU 对系统总线的操作是互斥的。CPU 不可能同时进行读写,读时不能写,写时不能读。

d) 同一器件的输入和输出是互斥的。例如,BX_{in},BX_{out}是互斥的。

e) 凡是有可能使状态引起竞争的控制信号都是互斥的。

一种性能良好而普遍使用的控制域编码方法是字段译码法,它是不译码法和译码法的结合。它将控制域分为若干字段。若各字段的编码相互独立,则通过各字段独立译码就可以获得计算机系统的全部控制信号,这被称做直接译码方式。若某些字段的编码相互关联,则关联字段要通过两级译码才能获得相关的控制信号,这被称做间接译码方式。字段编码结构如图 5-26 和图 5-27 所示。

在进行字段编码时,要遵循互斥的信号放在同一字段、相容的信号放在不同字段这一基本原则,以确保在字段编码后控制信号间有正确的相容、相斥关系。例如,控制信号 PC_{in}和 PC_{out}是程序计数器的输入与输出控制信号,如果两者同时有效,程序计数器的输入与输出

动作将发生冲突，造成程序计数器不能正常工作，所以 PC_{in} 和 PC_{out} 是互斥信号，编码时可放在同一字段，保证两者不会同时出现。而控制信号 AR_{in} 和 PC_{out} 是地址寄存器的输入锁存控制信号和程序计数器的输出允许控制信号，只有两者同时有效时，程序计数器的内容才能加载至地址寄存器中，实现微操作 PC →AR。所以，AR_{in} 和 PC_{out} 是相容信号，编码时应放在不同字段，保证两者可以同时在不同字段中出现。

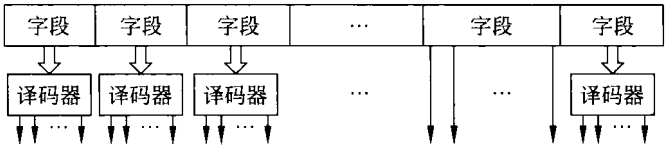


图 5-26　直接译码法的控制字段

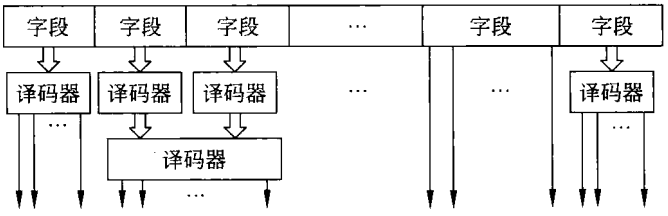


图 5-27　间接译码法的控制字段

在字段译码法中，若控制域有 L 个字段，则允许有 L 个控制信号同时有效。考虑到各字段间同时有效的控制信号可以有不同的组合，在每个字段中都要设计一个无效控制信号的编码。这样一来，若控制域的某字段有 m 位，则可以提供 2^m-1 个控制信号的编码。

在字段译码法中，如何组织字段对于微指令的长度和控制信号生成的时间有直接的影响。字段组织的有效方法有两种：按功能组织和按资源组织。

按功能组织就是把功能类同的各控制信号放在同一字段中。例如，针对如图 5-1 所示的 CPU，所有向内总线输出的微命令，如 PC_{out}、T_{out}、SP_{out}、AX_{out}、BX_{out} 等均为互斥信号，可放在同一字段中。对于器件的输入命令，如 PC_{in}、IR_{in}、SP_{in}、AX_{in}、AR_{in} 等均为 CPU 内部各寄存器的锁存输入控制信号，这些控制信号的功能相同，都是实现将 CPU 内部总线上的信息加载至相应寄存器内部的操作控制。这些控制信号中，凡是互斥信号，将其组织在同一字段中。其他功能相似的信号都可以按这样的思路归在一个字段中。

按资源组织就是把加载到同一部件上的各控制信号放在同一字段中。如前所述，为了保证多功能部件正常工作，部件都被设计成在一个时刻只实现一种功能。部件的控制信号必然是互斥的，可以组织在同一字段中。例如 ADD、SUB、AND、OR、XOR 等为控制 ALU 资源操作的控制信号，它们也是互斥的，应放在同一字段中。当然，这些控制信号也可以看做是一种按功能组织的结果，因为这些控制信号都用于实现运算类操作的控制。

(2) 垂直型微指令

垂直型微指令与水平型微指令的不同在于它们控制域的编码方法不同。

垂直型微指令的控制字段采用与机器指令相似的格式，由微操作码和微操作对象两大部分构成，其一般格式如图 5-28 所示。

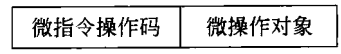


图 5-28　垂直型微指令格式

微指操作码用来指示做何种微操作,微操作对象用来为该微操作提供所需的操作数(常量或地址)。采用与机器指令相似的设计,微操作码也可以采用固定长度或可变长度进行编码,而微操作数可以有一个或多个。这种高度编码的方法,使垂直型微指令的控制域变得非常紧凑、短小,是减小微指令长度的有效设计方法。

垂直型微指令的控制字段一种极端情况是:如果CPU执行机器指令只需50余种微命令,则可用6位二进制编码构成微指令的控制字段。因为,一种编码只指定一种微命令,6位编码有64种,足以表示这50几种微操作命令。可以想象其构成的微指令一定是很短的。

每条垂直型微指令只能完成对少量微操作的控制,并行能力较差,致使微程序变长,执行速度减慢。改变这一状况的方法是在计算机系统中大量引入并行机制,使得少量的控制信号就可以引起较多的微操作同时完成。例如,采用单总线构造的ALU实现一次加法运算需要三个节拍、三条微指令来完成,采用双总线构造的ALU实现一次加法运算需要两个节拍、两条微指令来完成,而采用三总线构造的ALU实现一次加法运算仅需要一个节拍、一条微指令就可以完成了。

在这里还需说明,既然垂直型微指令就如同一条机器指令,前面已经看到,一条机器指令可用一段微程序实现。那么,一条微指令能否用一段毫微指令构成的毫微程序来完成呢?答案是肯定的。即将一条微指令分解成更细的毫微操作命令,设计毫微指令来实现毫微操作命令,用一段毫微指令构成的毫微程序来完成微指令的功能。具体细节不再讨论。

(3) 水平型与垂直型微指令的比较

水平型微指令具有如下特性。

① 具有位数较长的微指令,完成同样功能的微程序则比较短。

② 并行能力强、效率高。

③ 对控制域提供的控制信息只需较少的译码电路,甚至不需要译码。

④ 指令执行时间短。

⑤ 用户掌握、使用要困难一些。

垂直型微指令的特性与上述水平型微指令的特性相反。

4. 微指令的设计

在说明微指令控制域可采用水平型或垂直型编码的基础上,下面将以水平型编码具体构成如图5-1所示的CPU的微指令。

(1) 控制域字段

对CPU执行指令所需微命令,依据前面提到的按功能分为如图5-29所示的几个组成分组。

① 输出到内总线的命令

如图5-29中最左侧的一列,它们的作用是命令某个寄存器将其内容输出到内总线IB上。显然,这些信号一定是互斥的,这才将它们放在一个用4位编码的字段中。用这4位编码来表示这些微命令。若从0000到1011的编码顺序表示从NOP到T_{out},则PC_{out}微命令必为1001。该命令即可完成PC→IB微操作。

② 触发寄存器的命令

这是指XX_{in}微命令,它们也是互斥的,放在同一字段中。这些命令用这4位编码来表

示。同样，若从 0000 到 1100 的编码顺序表示从 NOP 到 T_{in}，则 PC_{in} 微命令必为 1001。该命令即可完成 IB→PC 微操作。

4b	4b	2b	2b	4b	3b	2b	2b	2b	2b
XX_{out}	XX_{in}	DR	AR	ALU	T	PC	M	SP	响应
NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP	NOP
AX_{out}	AX_{in}	DB→DR	AR→AB	ADD	SAR	0→PC	RD	SP+1	INTA
BX_{out}	BX_{in}	DR→DB	AR_{in}	ADC	SHL	PC+1	WR	SP−1	DACK
CX_{out}	CX_{in}			SUB	SHR				
DX_{out}	DX_{in}			SBC	ROL				
SP_{out}	SP_{in}			AND	ROR				
BP_{out}	BP_{in}			XOR	RCL				
SI_{out}	SI_{in}			NOT	RCR				
DI_{out}	DI_{in}			INC					
PC_{out}	PC_{in}			DEC					
DR_{out}	IR_{in}			CMP					
S_{out}	S_{in}			OR					
T_{out}	T_{in}								
	DR_{in}								

图 5-29　控制字段各类微命令分布

③ 其他微命令

图 5-29 中其他字段依据同样的思路，每一字段中的信号是互斥的，根据信号的多少决定该字段编码的位数。例如，AR_{in} 的编码应为二进制 10。

如图 5-29 所示的微指令控制域字段中，各字段的信号是相容的，可见该域的长度为 27 位。其中共有 54 个微命令，加一空操作为 55 个。若完全用垂直制微指令的思路编码，该控制域短，只需 6 位编码。

(2) 微指令的构成

前面已经把一条微指令的核心部分，控制域字段决定了，其形式如图 5-29 所示。

若控制存储器的容量为 512B，微指令采用一地址格式，则地址编码用 9 位即可表示。

后继地址控制字段 AC 设定为 4 位，4 位 AC 编码用于控制顺序执行、无条件转移、两分支转移和多分支转移。

这样一来，一条微指令的字长为 27＋9＋4＝40 位。微指令的格式如图 5-30 所示。

9b	4b	4b	4b	2b	2b	4b	3b	2b	2b	2b	2b
NA	AC	XX_{out}	XX_{in}	DR	AR	ALU	T	PC	M	SP	响应

图 5-30　微指令各组成字段格式

如图 5-30 所示，就可以构成各种所需要的微指令。利用所构成的微指令便可以编出对应机器指令的微程序。

(3) 时序

时序逻辑电路有两个作用。

① 依据时钟按节拍为控制存储器提供读出控制信号。将每个时钟周期作为一个节拍，在每个节拍发一次读出控制信号 read，从控存中读出并执行一条微指令，就可以实现一个节拍下的微操作。可见，在微程序控制器中，时序的设计与控制都很简单。

② 微程序运行时生成下一条微指令地址，并将其装入到微地址寄存器中。微指令地址域的设计不同，生成下一条微指令地址的硬件电路也会有所不同。

微程序控制器是一个相对而言比较简单的逻辑电路,它可以实现控制器应具备的功能:一是微指令(代表微操作)执行顺序的控制;二是控制信号的产生(引起微操作发生)。它在一个节拍周期内完成如下工作。

① 时序逻辑电路给控制存储器发出 read 命令。

② 从微地址寄存器 μAR 指定的控存单元读出微指令,送入微指令寄存器 μIR。

③ 根据微指令寄存器的内容,产生控制信号,给时序逻辑提供下条微地址信息。

④ 时序逻辑根据来自微指令寄存器的下条微地址信息和 CPU 内外状态,给微地址寄存器加载一个新的微地址。

微程序控制器依据节拍信号(以及触发信号,如图 5-6 所示)不断地重复这一工作过程,使存放在控制存储器中的微程序在机器指令的选择下一一被执行。显然,在微程序控制器中,时序控制相对比较简单。

5. 微程序编程

上面已经描述了微指令的构成,利用如图 5-30 所表示的微指令,能够实现产生执行机器指令所需要的任何控制命令和各种转移。接下来的问题就是如何编写微程序。

(1) 一条指令对应一段单独的微程序

为编写微程序,最易理解的就是对应 CPU 设计时所设置的所有指令,对应每一条指令单独编这条指令的微程序。将这些指令所对应的独立的微指程序,固化在 CM 中,CPU 执行哪条指令,就执行它所对应的那段微程序。

显然,这种方法很好理解,但并不适用。因为,如前所述,在每条指令所对应的单独的微程序中,有一些微程序段是重复的。最明显的就是前面小节中已经强调:所有指令取指令操作码的过程是一样的,为每条指令所共用。除此之外,还会有一些是多个微程序所共用的微程序段。若采用一条指令对应一段单独的微程序,显然是种浪费,更多的是用下一种方法。

(2) 共用可共用的微程序段

将那些可共用的微程序段单独编写出来,为那些可利用该段程序的指令使用。利用这种思路编写微程序的流程框图如图 5-31 所示。

在图 5-31 中,CPU 复位后即从启动地址取指令操作码,然后执行该指令。每执行完一条指令再取下一条指令操作码,这段取指令操作码的微程序段是共用的。

取得指令操作码后,根据这条指令的操作功能及寻址方式(也就是那条具体的指令)转到执行这条指令的微程序段上。如图 5-31 所示,由于指令功能及寻址方式不同,可以认为每一条指令的执行过程对应一段微程序。在执行过程的微程序段中,还有可能调用的共用微程序段,只是在图 5-31 上没有表示出来。

一条指令结束,需判断有无中断请求,有则响应并处理。在图 5-31 中暂未考虑 DMA 请求。

(3) 示例

下面首先编写共同使用的程序段,取指过程如下。

① PC_{out},AR_{in}

② AR→AB,RD,PC+1

③ MD→DB，DR_{in}

④ DR_{out}，IR_{in}

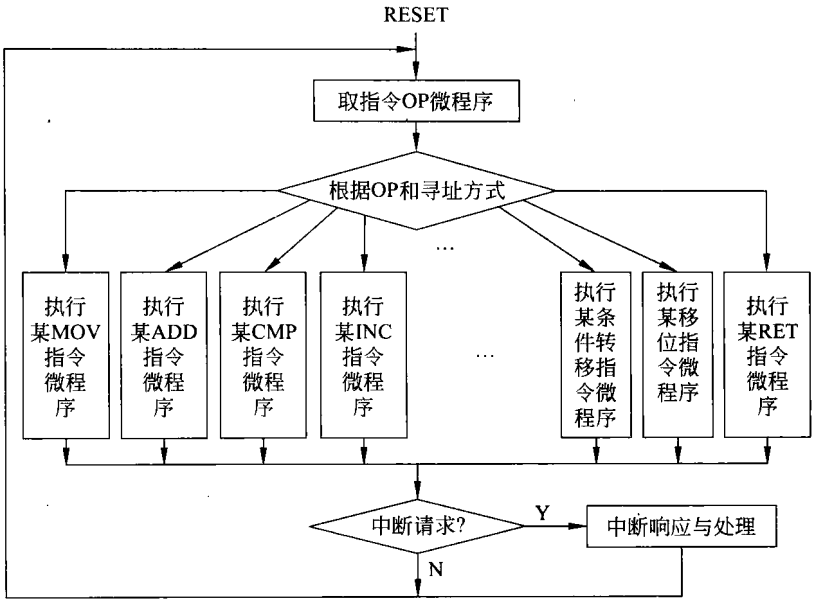


图 5-31　微程序流程框图

对应的 4 条微指令如图 5-32 所示。

NA	AC	XX_{out}	XX_{in}	DR	AR	ALU	T	PC	M	SP	响应
000000000	0000	1001	0000	00	10	0000	000	00	00	00	00
000000000	0000	0000	0000	00	01	0000	000	10	01	00	00
000000000	0000	0000	1101	01	00	0000	000	00	00	00	00
000000000	1001	1010	1010	00	00	0000	000	00	00	00	00

图 5-32　取指令操作码的微程序段

在取指过程中，AC 均为 0，表明是顺序执行的。在此过程中，后继地址是不用的，故 9 位全为 0。控制域的字段为 0 表示空操作。

取指过程对任何指令都是适用的。当指令操作码取出并放入 IR 后，后面的执行过程将因指令不同而异。下面仍以 ADD　BX，[DI]为例说明指令执行过程的微程序段。

利用如图 5-32 所示的微程序段即可将指令 ADD　BX，[DI]的指令码（如前所述包括寻址方式字节在内的 16 位）放到了指令寄存器 IR 中。

图 5-32 中的最后一条微指令将指令码放入 IR 中。同时，微指令中的 AC（此处为 1001）指出如何形成 NA。该字段控制如图 5-24 所示形成该指令执行微程序入口地址：根据寄存器间接寻址的加法指令 ADD　BX，[DI]的指令码，从 ROM1 中查出 NA 的基地址（例如 080H），根据指令的寻址方式，从 ROM2 中得到位移量（假定为 07H），将两者加到一

起就形成了该指令的目的地址(087H)。从此地址开始,可存放执行该指令的微程序。

执行步骤如下。

① DI_{out},AR_{in}

② AR→AB,RD

③ MD→DB,DR_{in}

④ DR_{out},S_{in}

⑤ S_{out}→ALU

⑥ BX_{out}→ALU

⑦ ADD→T,T_{in}

⑧ T_{out},BX_{in}

上述过程编写微程序如图 5-33 所示。

NA	AC	XX_{out}	XX_{in}	DR	AR	ALU	T	PC	M	SP	响应
000000000	0000	1000	0000	00	10	0000	000	00	00	00	00
000000000	0000	0000	0000	00	01	0000	000	00	01	00	00
000000000	0000	0000	1101	01	00	0000	000	00	00	00	00
000000000	0000	1010	1011	00	00	0000	000	00	00	00	00
000000000	0000	1011	0000	00	00	0000	000	00	00	00	00
000000000	0000	0010	0000	00	00	0000	000	00	00	00	00
000000000	0000	0000	1101	00	00	0001	000	00	00	00	00
110000000	0001	1100	0010	00	00	0000	000	00	00	00	00

图 5-33 ADD BX,[DI]指令执行过程的微程序

需要说明的是,上述微程序是顺序程序,由上到下顺序执行即可产生相应的微控制命令,最后将相加的结果放在 BX 中。在这过程中 AC 均为 0,表示为顺序执行,这时 NA 中的地址是不用的。这段微程序就放在首地址为 087H 的顺序单元中,此后每一次执行这条指令都执行这一段程序。

但在最后一条微指令中,AC 的编码为 0001,这表示无条件转移,转移地址为 NA 的内容。在这里假定上面取指微程序的首地址就是 000000000。则本指令执行结束就会无条件转移到检测中断请求的共用微程序其 NA 地址为 180H,若有中断请求则转向中断响应;若无中断请求则转向取指令共用程序,去取下一条指令的指令码。(此处假定不考虑 DMA 的请求)

以上仅以一条指令为例来说明微程序的编程。由于作为描述对象的 CPU 设置了多种指令和寻址方式,因此,微程序设计相对也就比较麻烦。但是,有如图 5-31 所示的思路及上

面的例子，读者能理解微程序控制器的原理及微程序的设计思路即可。

6. 微程序控制器与硬布线控制器的比较

硬布线控制器完全利用逻辑器件实现，所以速度非常快。当计算机系统越来越复杂时，硬布线控制器设计也越来越困难。硬布线控制器一旦被实现，便不可修改和扩充。

微程序控制设计使控制器设计简单化、规范化，并使其功能具有可修改性和可扩充性。微程序控制器内部的译码和时序电路非常简单，实现成本低，出错概率小。微程序控制器的主要缺点是比硬布线控制器速度慢。

对于功能复杂的计算机系统，微程序控制器相对易于实现，所以 CISC 处理器几乎都采用微程序控制器。而 RISC 处理器具有简单的指令格式，又强调高速度，所以主要采用硬布线控制器。

习　题

1. 中央处理器(CPU)由哪些部分组成？它们应具有哪些基本功能？
2. 控制器的功能是什么？它由哪些部件组成？
3. 什么是硬布线控制器？有什么特点？
4. 硬布线控制器如何产生微命令？产生微命令的主要条件有哪些？
5. 什么是指令周期和微指令周期？二者之间存在什么关系？
6. 微程序控制的基本思想是什么？微程序控制器的特点是什么？
7. 微程序控制器设计中微指令由哪几部分组成？各部分的作用是什么？
8. 微指令产生控制信号(微命令)的方式有哪些？各方式是如何产生控制信号的？
9. 对微指令进行字段直接编码的基本原则是什么？
10. 主存与控存有什么异同？
11. 与硬布线控制器相比，微程序控制器有哪些优缺点？
12. 在微程序控制器设计中，若控制器需要处理 100 条指令，每条指令需要最多 8 条微指令构成的微程序来实现其功能；控制器需要产生 100 个微命令(控制信号)，且被分为 6 个相容类，如表 5-1 所示。

表 5-1　微命令相容类表

类型	0	1	2	3	4	5
微命令数	14	29	8	23	11	15

(1) 请合理设计微指令的控制域(微命令生成字段)，使其长度尽可能的短。

(2) 若微指令的次地址(即下一条微指令地址)产生仅是顺序地址和无条件跳转地址，请合理设计微指令的地址域(下条微指令地址生成字段)。

(3) 请根据以上设计确定控制存储器多大为宜？

13. 某微程序控制器采用的微指令字长为 24 位。微命令生成部分由 4 个字段构成，各字段所包括的互斥微命令分别为 5 个、8 个、14 个和 3 个。另外控制产生次地址的条件有 3

种。试说明该微控制器最多可用几位来表示次地址？控制存储器的容量为多少？

14. 某计算机系统简化的 CPU 结构如图 5-1 所示，实现指令 ADD AX，BX(功能为(AX)+(BX)→AX)时采用以下微操作流程。

取指令：

```
PC→AR
AR→AB,RD,PC+1
DB→DR
DR→IR
```

执行指令：

```
AX→S
BX→IB,ADD
ALU→T
T→AX
```

(1) 依据此流程，写出实现加法指令 ADD AX，(BX)的微流程。说明：该指令中(BX)为寄存器间接寻址，指令功能为(AX)+((BX))→AX，RD 为存储器读控制信号，ADD 为运算器加法控制信号。

(2) 试拟出加法指令 ADD AX，DISP(SI)的微流程。其中 DISP(SI)表示变址寻址，DISP 是偏移量，SI 是变址寄存器。

(3) 拟出减法指令 SUB BX，100(DI)的指令微流程。

15. 图 5-34 为某一运算器的简化框图，其中 A、B 既有寄存器又有多路选择器的功能，M 有移位和选择器的功能，ALU 为算术逻辑单元，R0 和 R1 为通用寄存器。图中粗线为数据通路，是一种单总线结构。带下标 in 者为数据存入该寄存器的微命令，带下标 out 者为将该寄存器的内容输出的微命令，XX→ALU、ADD、+1、V、L 等均为微命令。

(1) 根据图中所示，选择括号中的正确答案。

① $R0_{out}$，A_{in}(相容/互斥)　② V，R，L(相容/互斥)

③ $R0_{out}$，$R1_{out}$(相容/互斥)　④ A→ALU，ADD，V(相容/互斥)

⑤ ADD，AND(相容/互斥)

(2) 若 2((R0)−1)→R0 的微命令序列如下所示。

① $R0_{out}$，B_{in}，　；(R0)→B

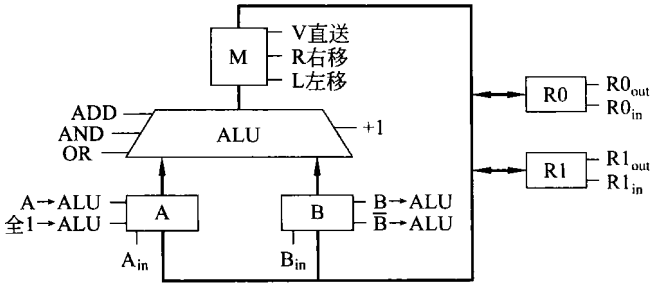


图 5-34　习题 15 附图

② B→ALU，全 1→ALU，ADD，L，$R0_{in}$ ；((B)－1)×2→R0

试写出执行(R0) OR (R1)→R0 所需的微命令序列。

16. 某假想主机的主要部件如图 5-35 所示。其中 R0～R3 为通用寄存器，A、B 为暂存器，其他部件如图中所标。

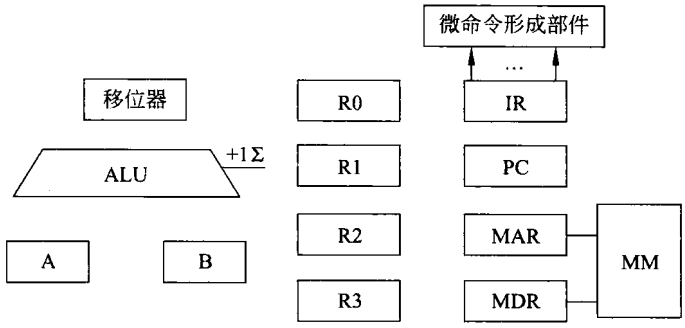


图 5-35 习题 16 附图

(1) 在上图的基础上，画出其内部的数据通路并标明数据流向。

(2) 写出传送指令 MOV R0，R1(即将 R1 的内容传送到 R0)的指令微操作流程。

17. 某 CPU 结构如图 5-36 中虚框内所示，其中包括一个累加寄存器 AC、一个状态寄存器和其他 4 个寄存器，各部分之间的连线表示数据通路，箭头表示信息传送方向。

(1) 图中的 4 个寄存器 A、B、C、D 分别是什么功能的寄存器？

(2) 写出 LDA X 指令执行阶段的微操作流程(X 为内存地址，LDA X 功能为(X)→AC)。

(3) 写出 STA Y 指令执行阶段的微操作流程(Y 为内存地址，STA Y 功能为(AC)→(Y))。

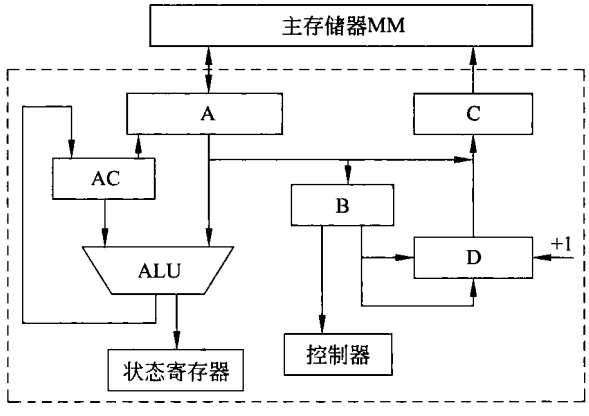


图 5-36 习题 17 附图

18. 已知某假想机的运算器逻辑如图 5-37 所示，它的核心部分是一个并行加法器，S 为加法器的输出，加法器的输入为寄存器 A 和 B，最低进位信号为'＋1'('＋1'为高电平时，进位为 1)。图中各带有下标 in 的控制信号为寄存器(A，B，C，D)的信息打入脉冲，其他为节拍控制电位信号，节拍控制电位与打入脉冲的相位关系如图 5-37 右下角所示。

写出实现运算(D)－1→(D)的微操作序列。

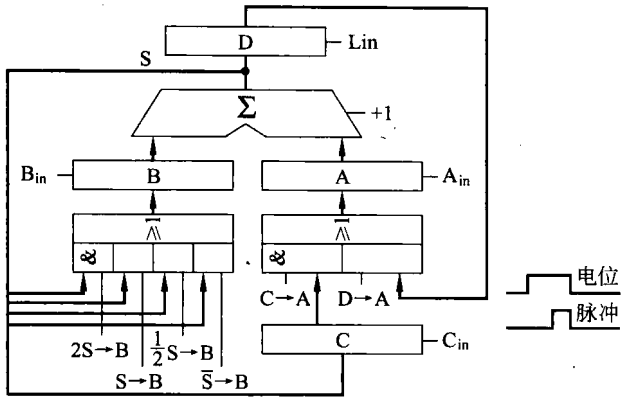


图 5-37　习题 18 附图